



Data Structures and Algorithms

Geetha Lekshmy V

Harsha Raj

Department of Computer Science

Amrita Vishwa Vidyapeetham

Objectives

Binary Search Tree

Tree Traversal

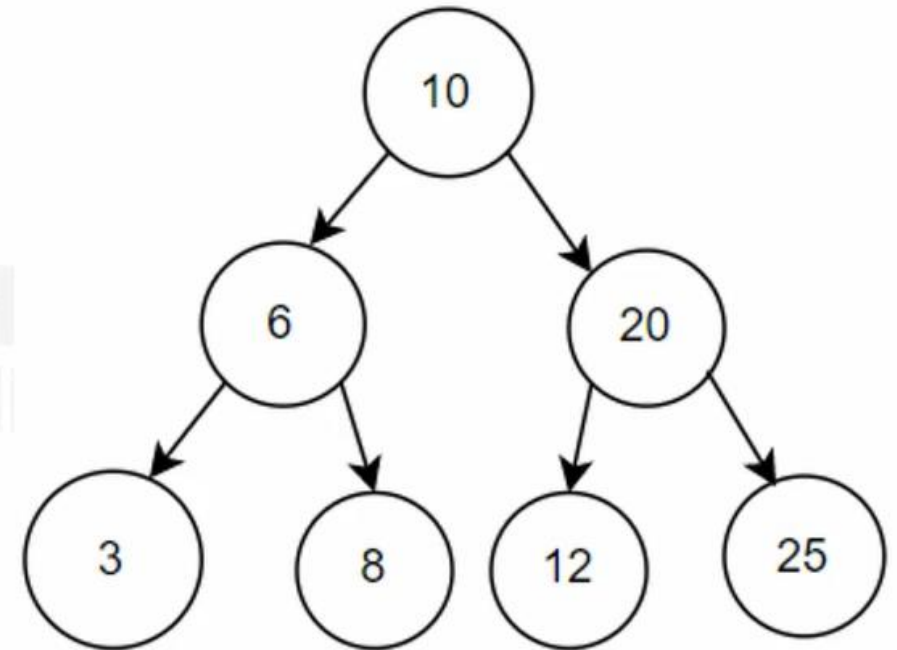


Place your
Webcam Video here
Size 100%

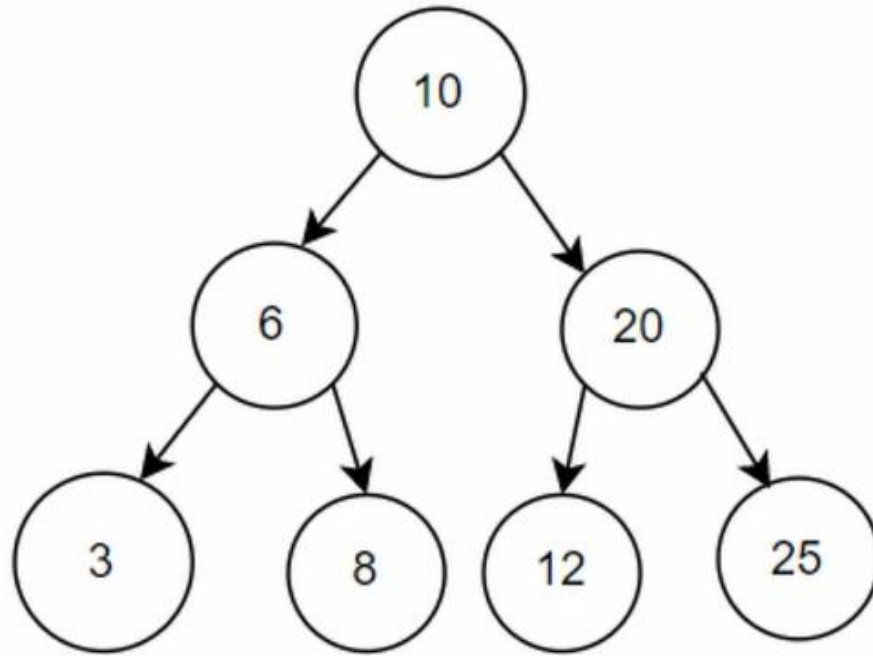


Binary Search Tree

- Ordered Binary Tree.
- Keys in left subtree are less than root.
- Keys in right subtree are greater than root.
- Left and Right subtree should also be binary search trees.
- Binary search trees are used in search applications, where data is updated frequently.

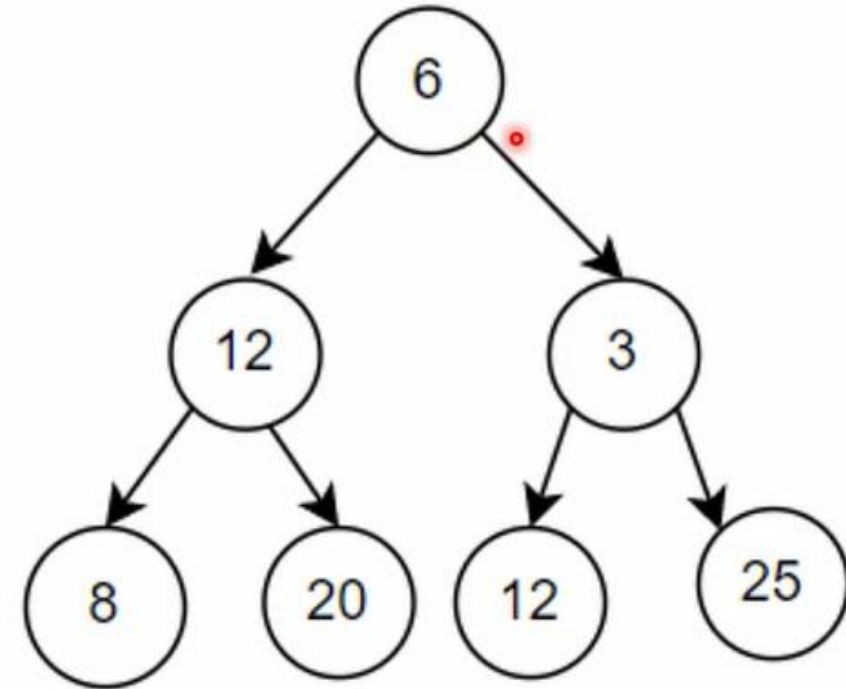


Binary Search Tree



Time complexity of search - $O(\log n)$

Binary Tree



Time complexity of search - $O(n)$

Tree traversals



- Visit each node of the tree and perform operations, say print the content.
- Pre-order Traversal (Root, Left subtree, Right subtree)
- Post-order Traversal (Left subtree, Right subtree, Root)
- Inorder Traversal (Left subtree, Root, Right subtree)
- Level order Traversal (visit the tree level by level starting from level 0)



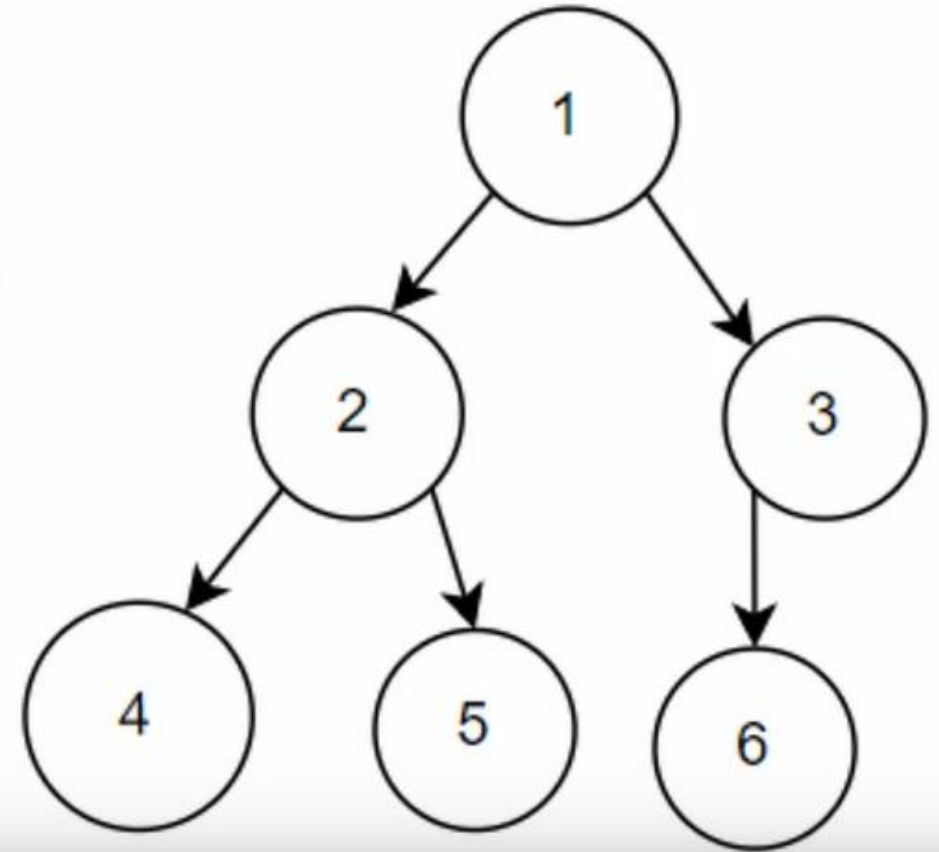
Tree Traversals

Pre-order – Root, Left, Right
1 2 4 5 3 6

Post-order – Left, Right, Root
4 5 2 6 3 1

Inorder – Left, Root, Right
4 2 5 1 6 3

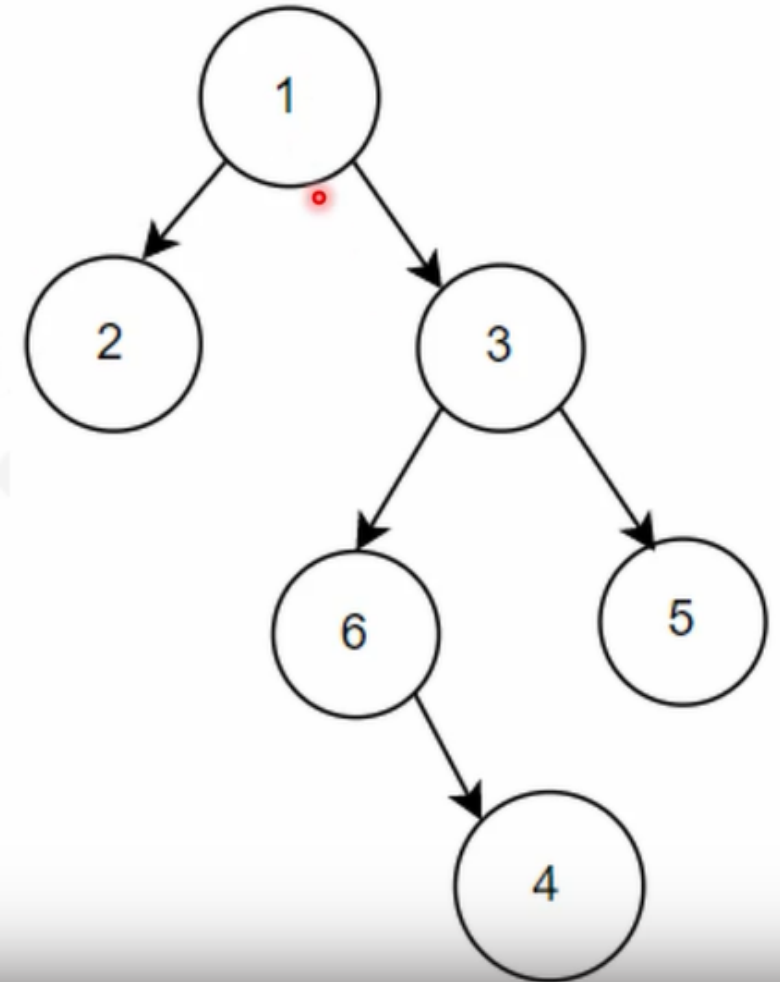
Level order
1 2 3 4 5 6



Tree Traversal - Pre-order

- Visit the root
- Visit left subtree in pre-order.
- Visit right subtree in pre-order.

What is the order in which the nodes are visited in this tree?

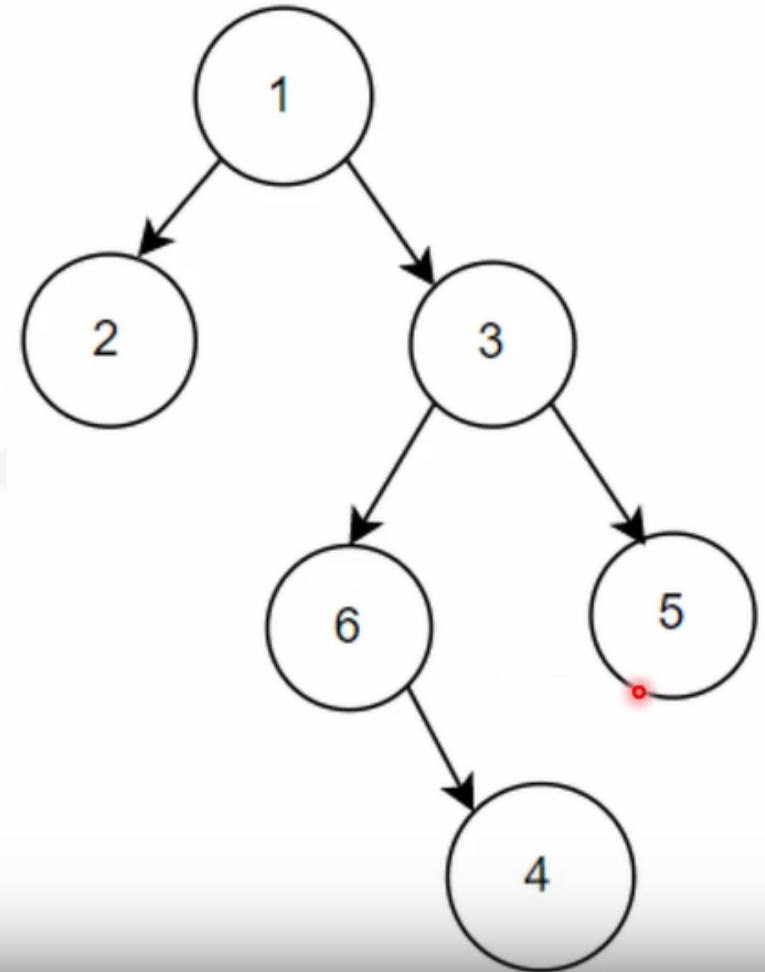


Tree Traversal - Pre-order

- Visit the root
- Visit left subtree in pre-order.
- Visit right subtree in pre-order.

What is the order in which the nodes are visited in this tree?

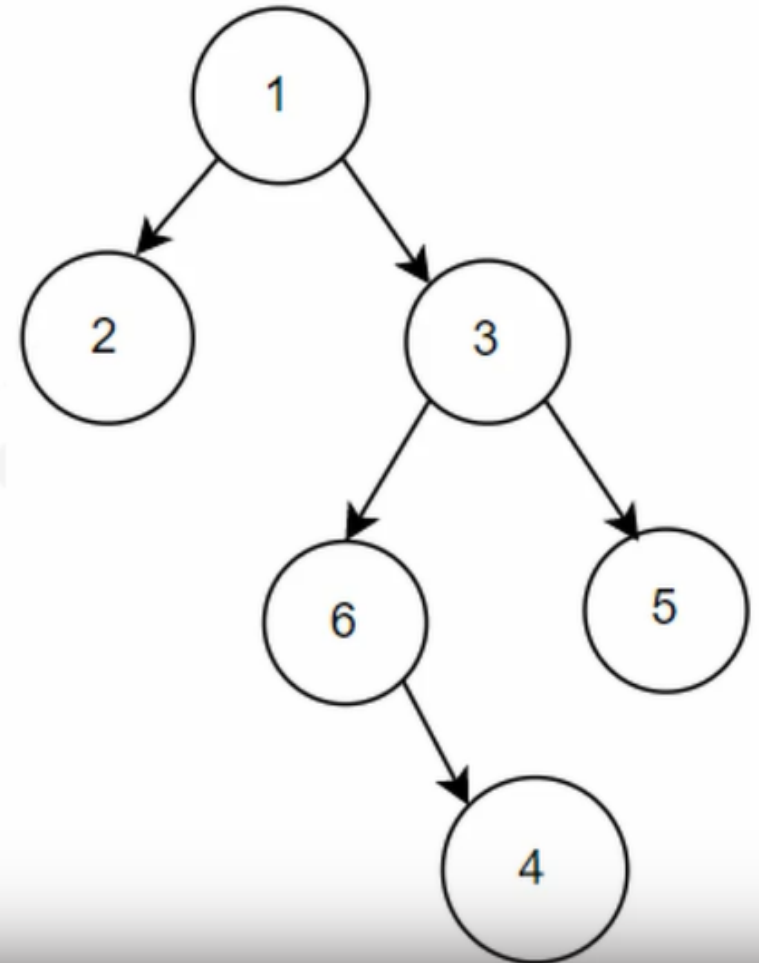
1 2 3 6 4



Tree Traversal - Post-order

- Visit left subtree in post-order.
- Visit right subtree in post-order.
- Visit the root

What is the order in which the nodes are visited in this tree?

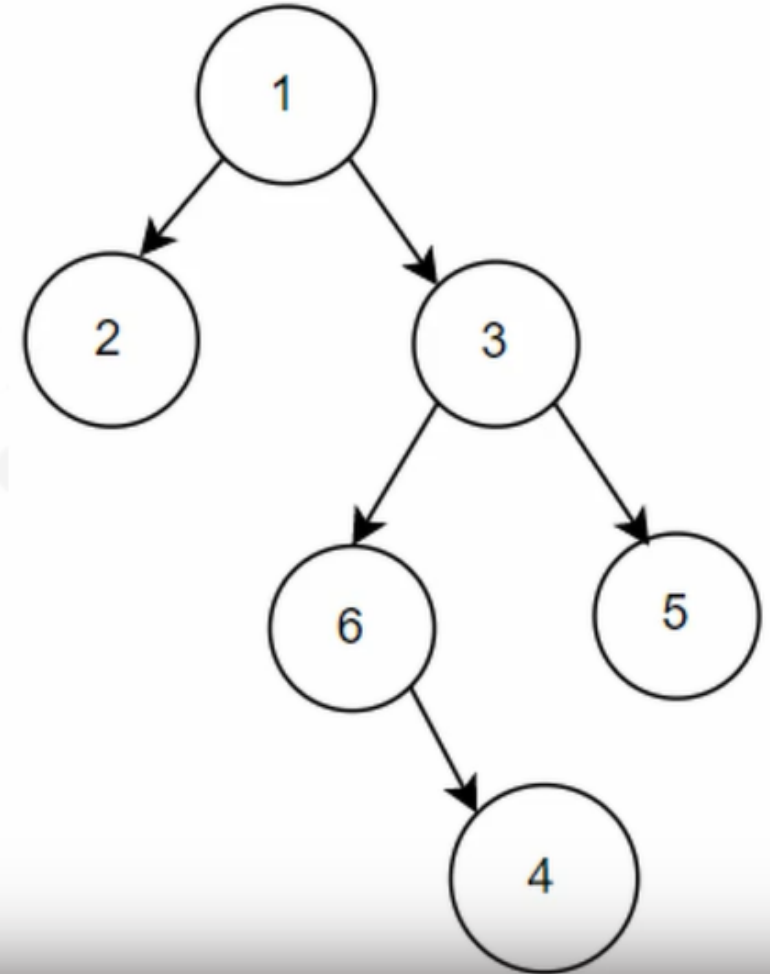


Tree Traversal - Post-order

- Visit left subtree in post-order.
- Visit right subtree in post-order.
- Visit the root

What is the order in which the nodes are visited in this tree?

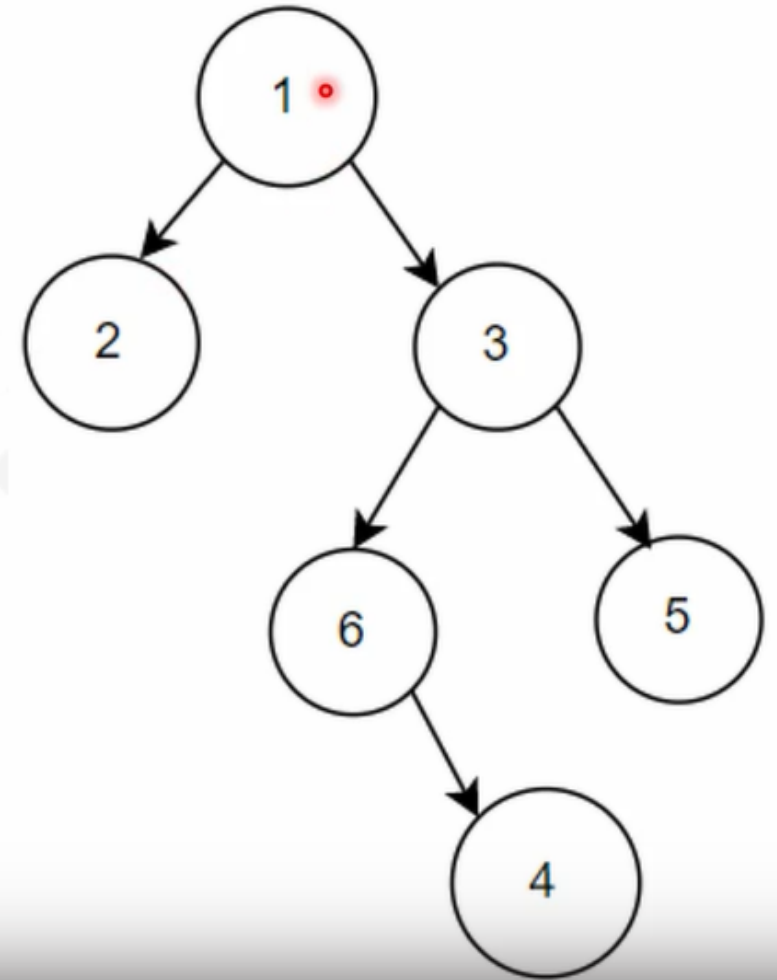
2 4 6 5 3 1



Tree Traversal - In-order

- Visit left subtree in in-order.
- Visit the root
- Visit right subtree in in-order.

What is the order in which the nodes are visited in this tree?

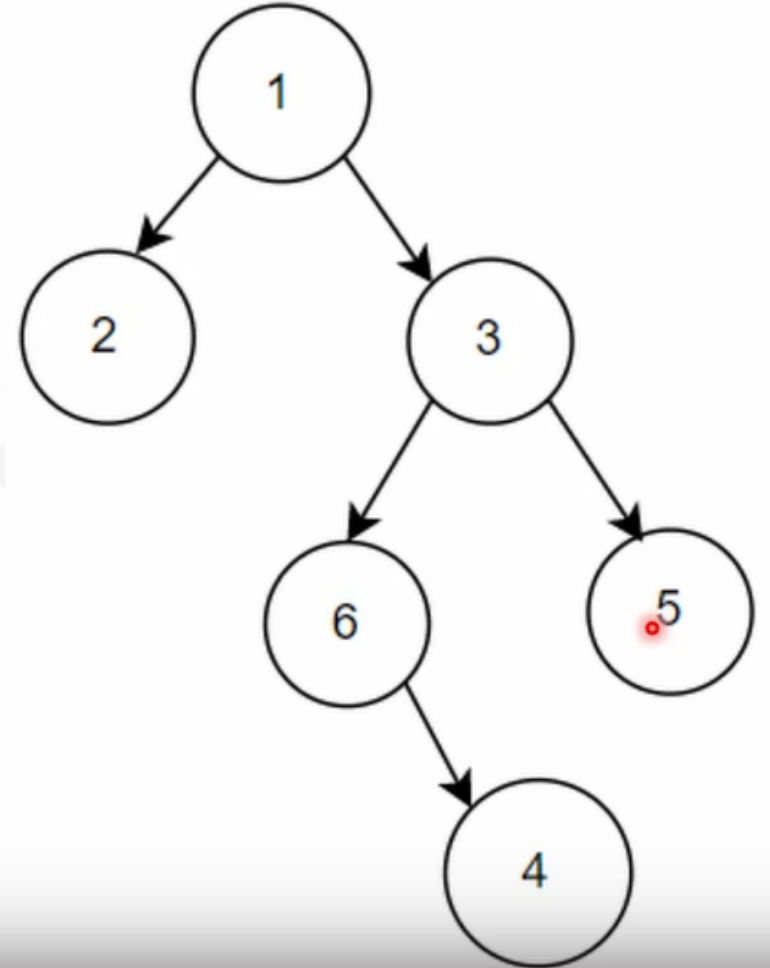


Tree Traversal - In-order

- Visit left subtree in in-order.
- Visit the root
- Visit right subtree in in-order.

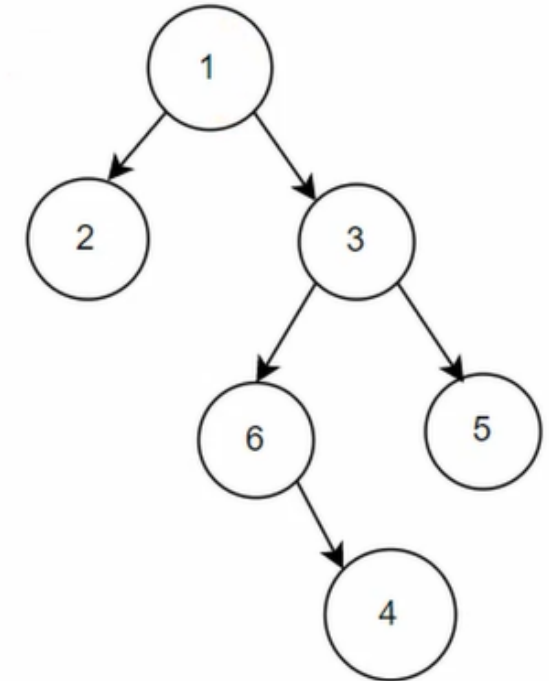
What is the order in which the nodes are visited in this tree?

2 1 6 4 3 5



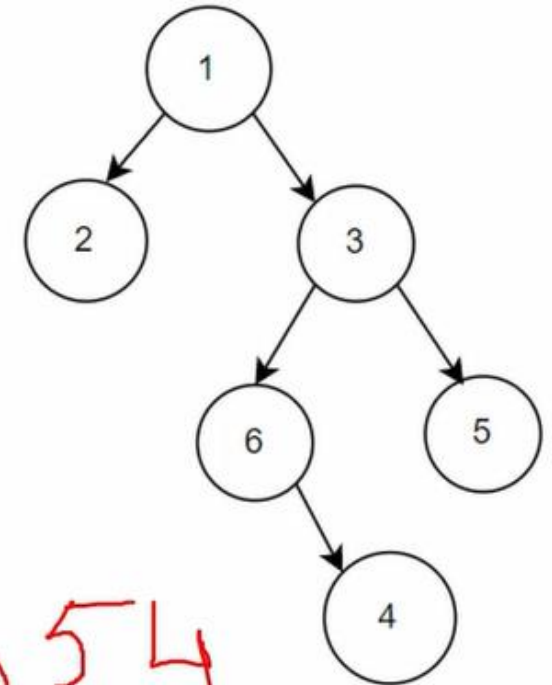
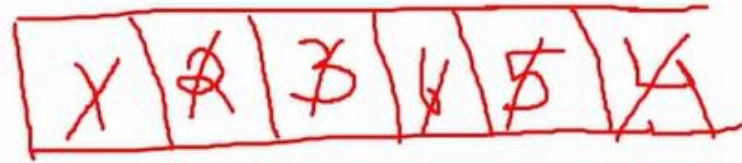
Tree Traversal - Level-order

```
Queue Q;  
Q.createQueue()  
Visit the root.  
Q.Enqueue(root)  
While( !Q.isEmpty()){  
    t = Dequeue()  
    print(t.getData());  
    if(t.getLeft()!=null)  
        Q.enqueue(t.getLeft())  
    if(t.getRight()!=null)  
        Q.enqueue(t.getRight())  
}
```



Tree Traversal - Level-order

```
Queue Q;  
Q.createQueue()  
Visit the root.  
Q.Enqueue(root)  
While( !Q.isEmpty()){  
    t = Dequeue()  
    print(t.getData());  
    if(t.getLeft()!=null)  
        Q.enqueue(t.getLeft())  
    if(t.getRight()!=null)  
        Q.enqueue(t.getRight())  
}
```

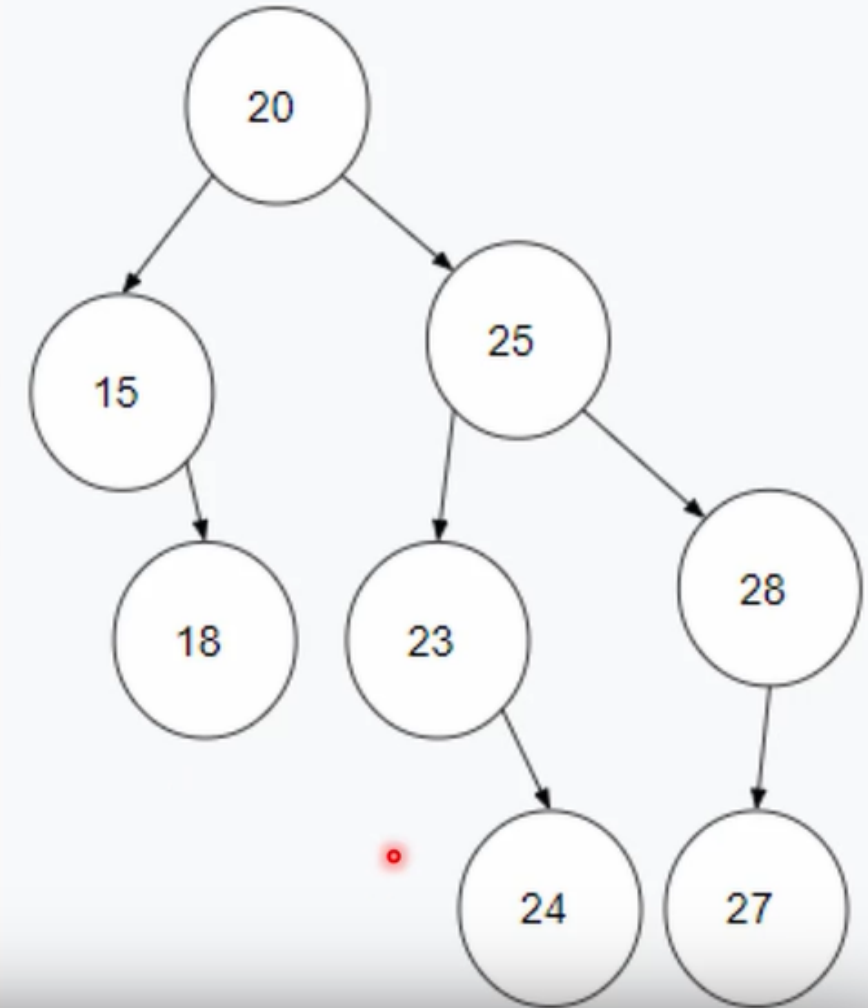


1 2 3 6 5 4

Tree Traversal – Binary Search Tree

Determine the order in which the nodes are visited in

- Inorder
- Preorder
- Postorder
- Level order

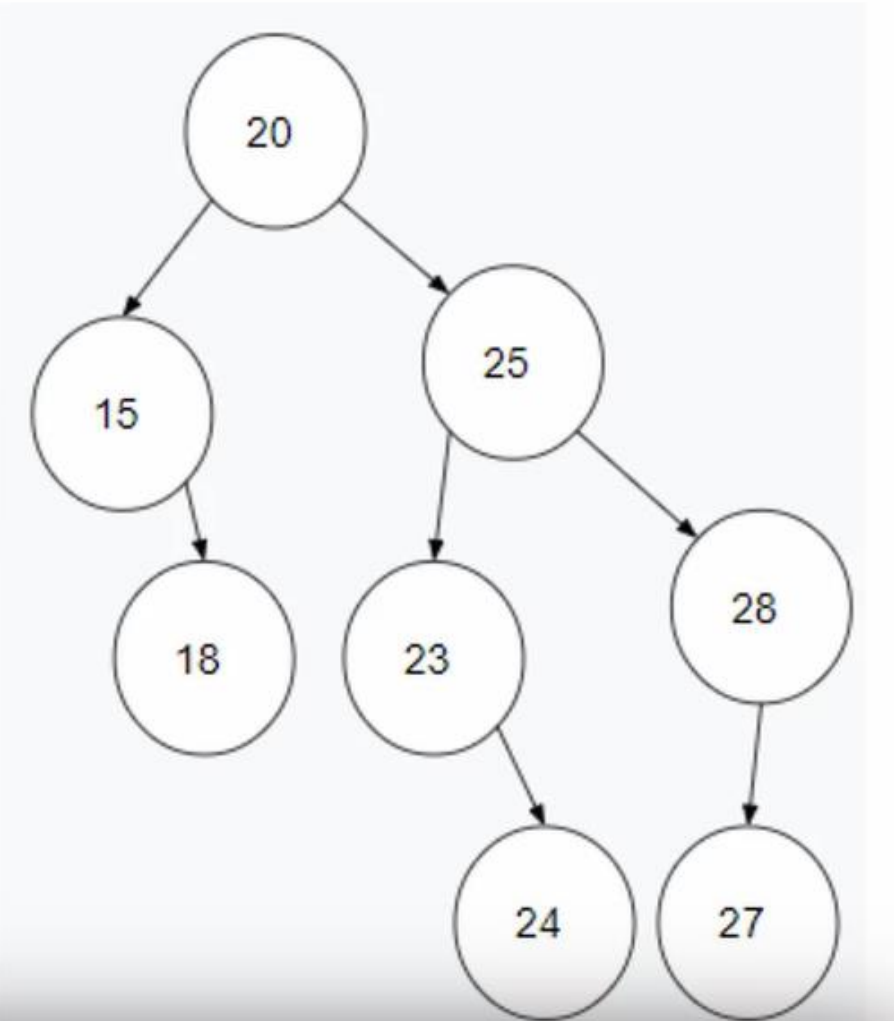


Tree Traversal – Binary Search Tree

Determine the order in which the nodes are visited in

Sorted

- Inorder 15 18 20 23 24 25 27 28
- Preorder 20 15 18 25 23 24 28 27
- Postorder 18 15 24 23 27 28 25 20
- Level order 20 15 25 18 23 28 24 27



Summary

Binary Search Trees – ordered arrangement of data

Inorder, Preorder Postorder and level order traversals



AHEAD Online

Data Structures and Algorithms

Geetha Lekshmy V

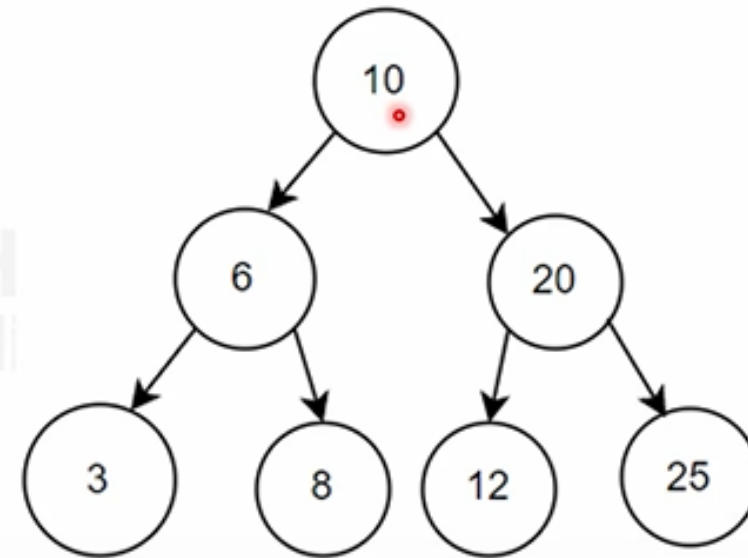
Harsha Raj

Department of Computer Science

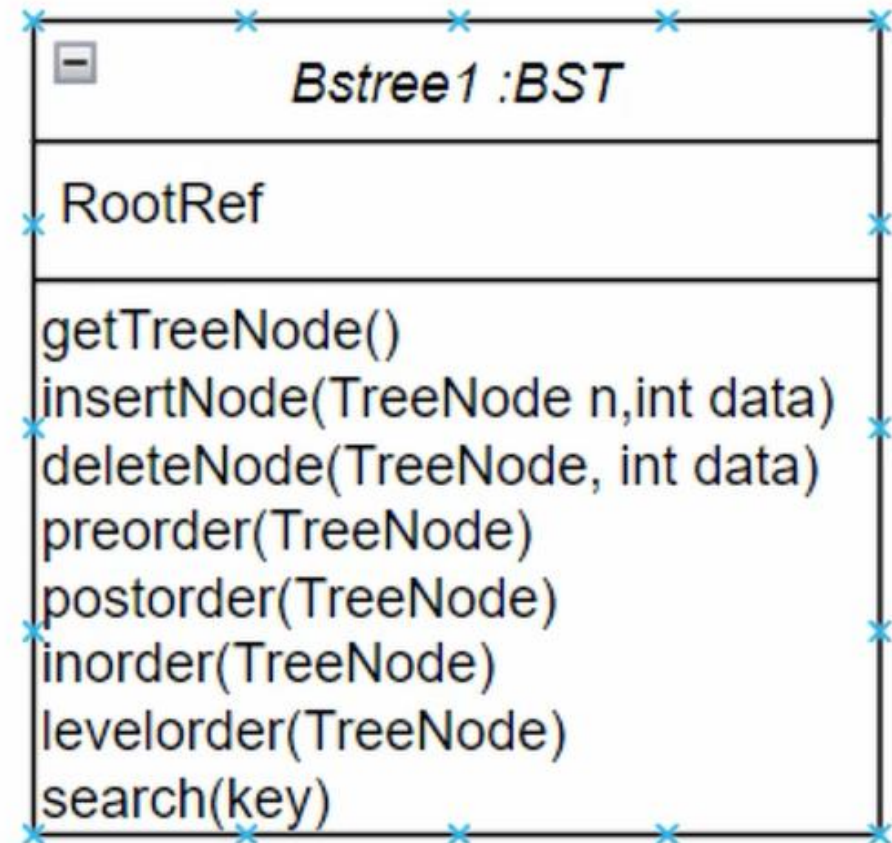
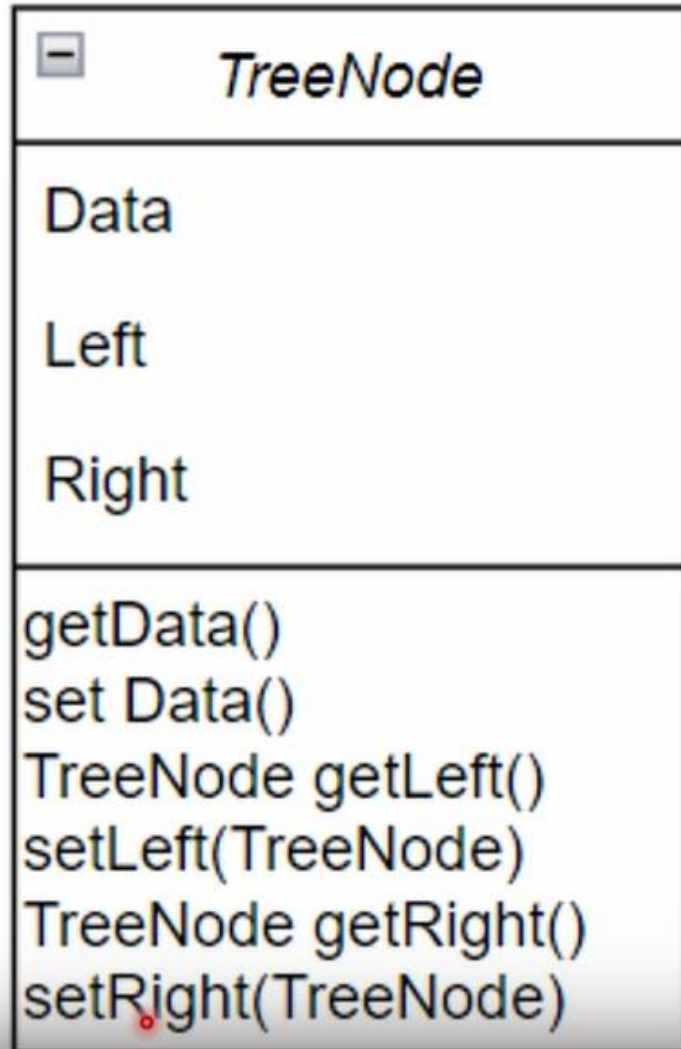
Amrita Vishwa Vidyapeetham

Binary Search Tree

- Keys in left subtree are less than root.
- Keys in right subtree are greater than root.



Binary Search Tree – Node, Tree



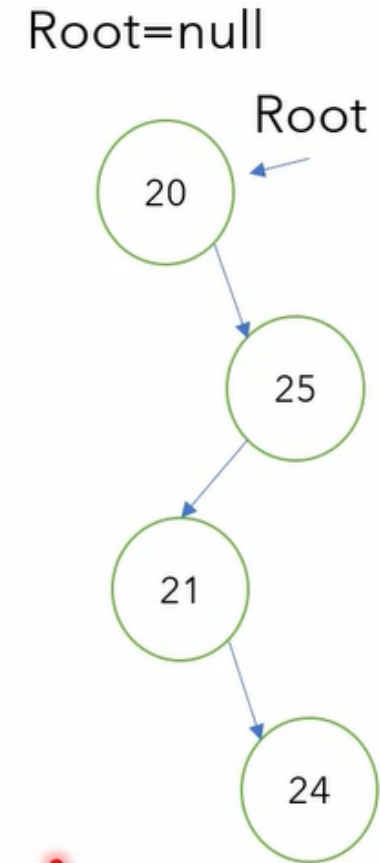
Binary Search Tree – Insertion (Recursive)

Root=null

```
TreeNode insertNode(TreeNode n, int d ) {  
    if (n==null) { return new TreeNode(d); }  
    else {  
        if(d < n.getData()) {  
            TreeNode newn= insertNode(n.getLeft(),d);  
            n.setLeft(newn);  
        }else{  
            TreeNode newn= insertNode(n.getRight(),d);  
            n.setRight(newn);  
        }  
    }  
    return n;  
}
```

Binary Search Tree – Insertion (Recursive)

```
TreeNode insertNode(TreeNode n, int d ) {  
    if (n==null) { return new TreeNode(d); }  
    else {  
        if(d < n.getData()) {  
            TreeNode newn= insertNode(n.getLeft(),d);  
            n.setLeft(newn);  
        }else{  
            TreeNode newn= insertNode(n.getRight(),d);  
            n.setRight(newn);  
        }  
    }  
    return n;  
}
```



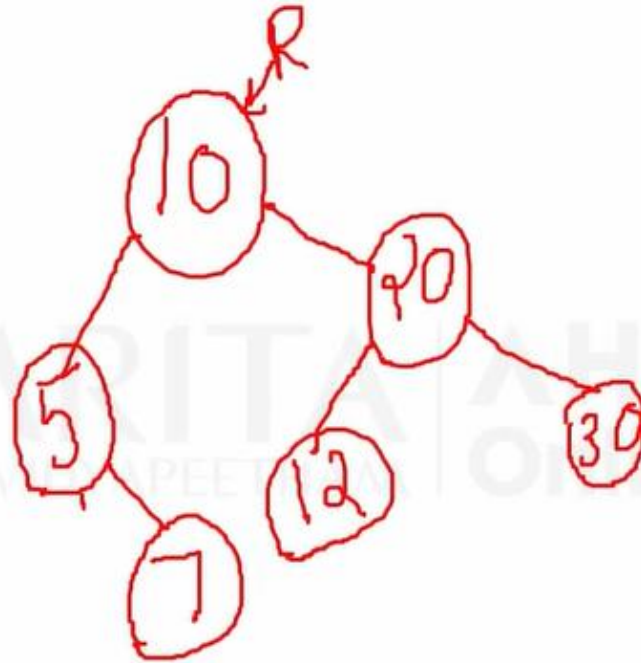
Binary Search Tree – Insertion

Insert the following keys into a Binary Search Tree - 10, 20, 30, 12, 5, 7



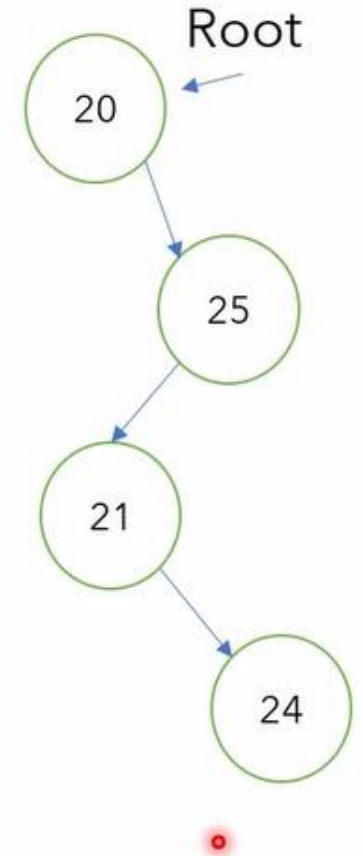
Binary Search Tree – Insertion

Insert the following keys into a Binary Search Tree - 10, 20, 30, 12, 5, 7



Binary Search Tree – search

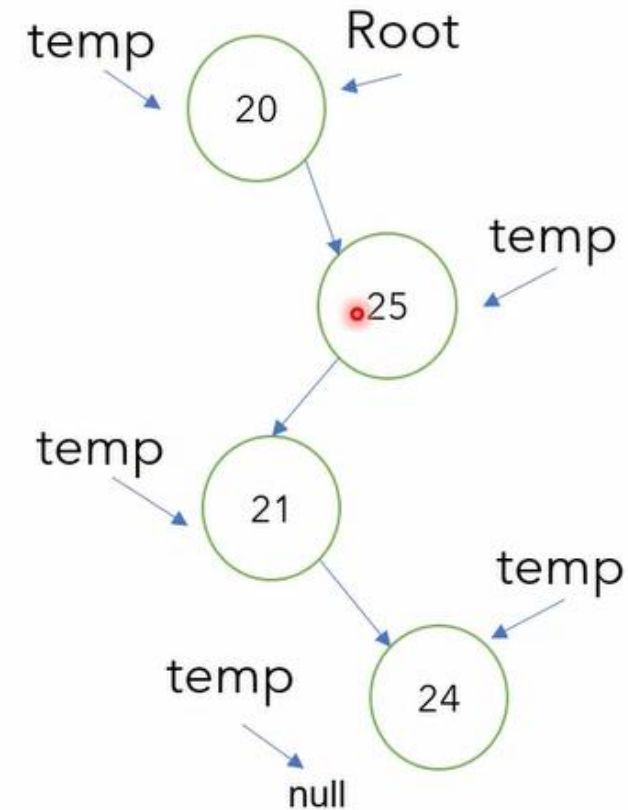
```
public boolean search(int key) {  
    TreeNode temp=root;  
    while(temp!=null) {  
        if(temp.getData() == key) {return true;}  
        else if(key < temp.getData()) {  
            temp=temp.getLeft();  
        }  
        else {temp=temp.getRight();}  
    }  
    return false;  
}
```



Binary Search Tree – search

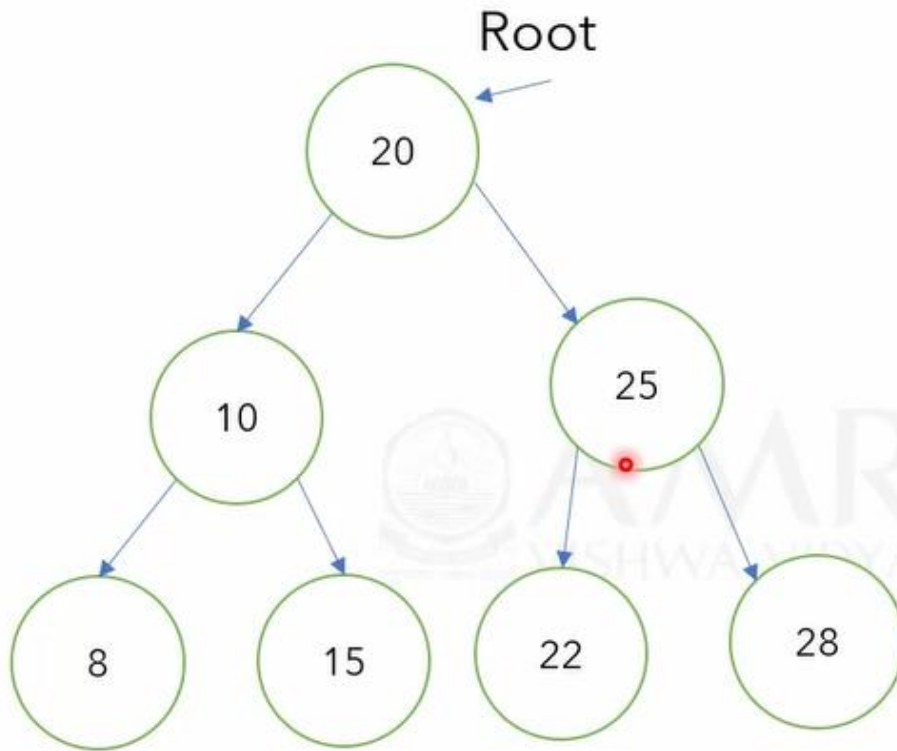
```
public boolean search(int key) {  
    TreeNode temp=root;  
    while(temp!=null) {  
        if(temp.getData() == key) {return true;}  
        else if(key < temp.getData()) {  
            temp=temp.getLeft();  
        }  
        else {temp=temp.getRight();}  
    }  
    return false;  
}
```

K=23



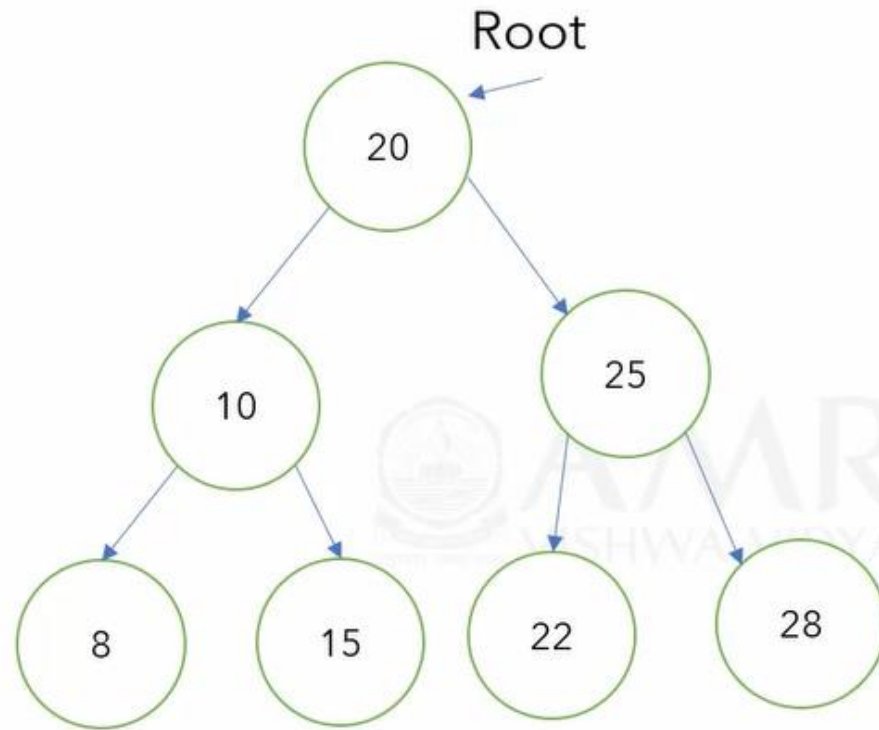
Binary Search Tree – search

How many comparisons are required for searching key 28 from this tree?



How many comparisons for searching a key from a complete binary search tree with 'n' nodes?

Binary Search Tree – search



How many comparisons are required for searching key 28 from this tree?



How many comparisons for searching a key from a complete binary search tree with 'n' nodes?



Summary

Binary Search Tree – Insertion and Search



VISHWA VIDYAPEETHAM

AHEAD
Online





AHEAD Online

Data Structures and Algorithms

Geetha Lekshmy V

Harsha Raj

Department of Computer Science

Amrita Vishwa Vidyapeetham

Objectives

Binary Search Tree

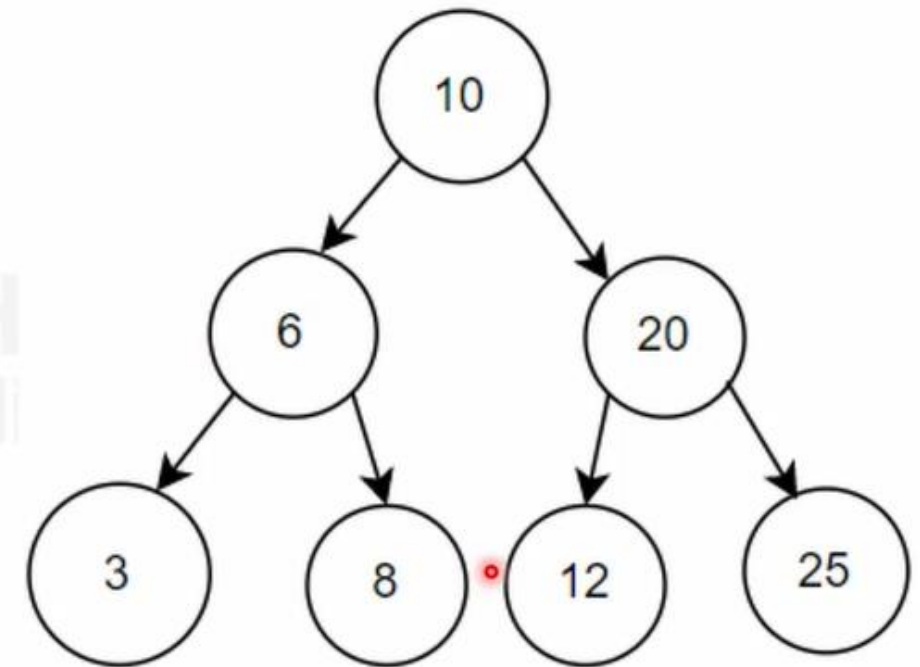
deletion



Place your
Webcam Video here
Size 100%

Binary Search Tree

- Keys in left subtree are less than root.
- Keys in right subtree are greater than root.



Binary Search Tree – deletion

Step 1

Locate the node to be deleted in the tree.

Step-2

Check the node to be deleted

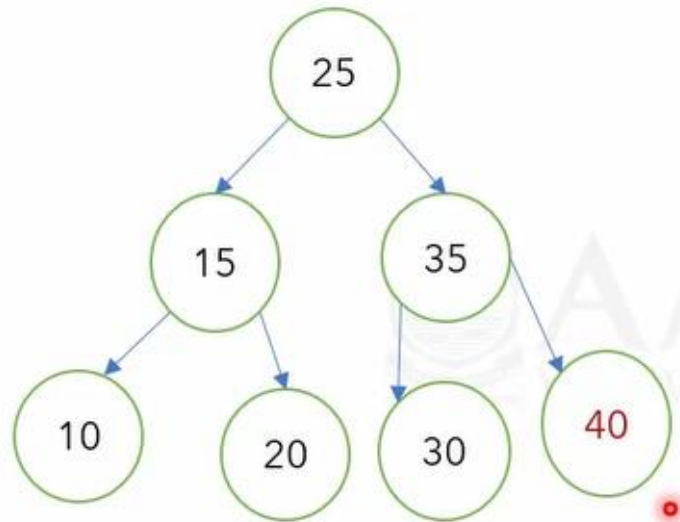
1. is a leaf – set the reference of the parent to null (if leaf is a left child of parent set left reference of parent to null)
2. has one child, copy the content of child to current node and delete child.
3. has two children. Replace the content of the node with the content of inorder successor / inorder predecessor and delete inorder successor / inorder predecessor.

Binary Search Tree – deletion

Node to be deleted – Leaf Node

Node to be deleted -40

Search the node and set the left/right child its parent to null.

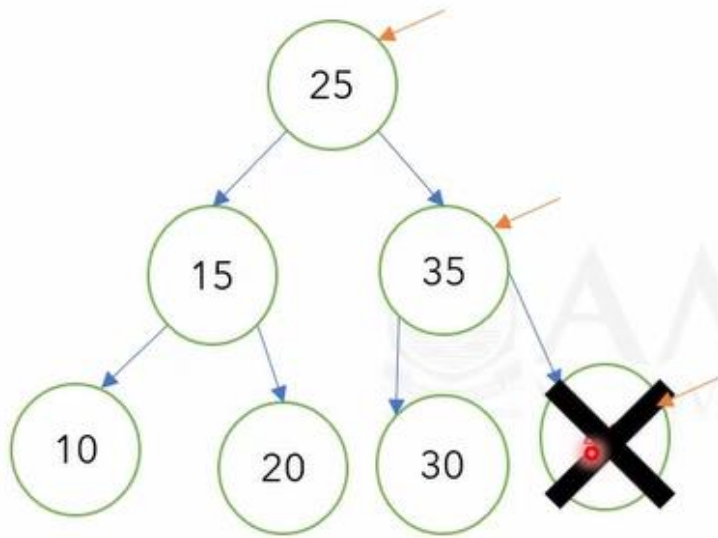


Binary Search Tree – deletion

Node to be deleted – Leaf Node

Node to be deleted -40

Search the node and set the left/right child its parent to null.

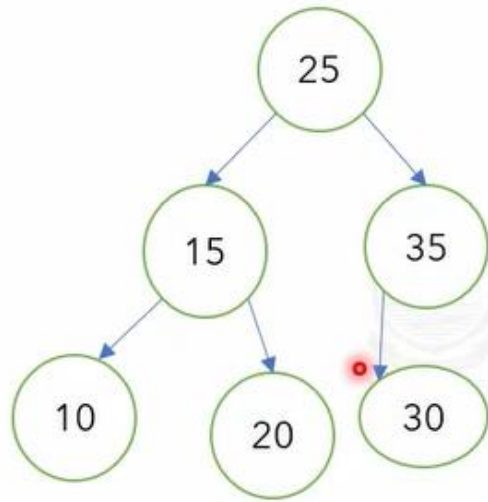


Binary Search Tree – deletion

Node to be deleted – Node with single child

Node to be deleted -35

Search the node and copy the content of the child into parent and delete child.

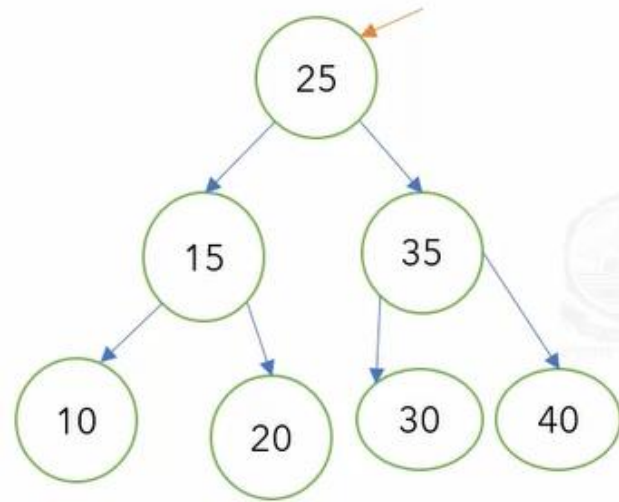


Binary Search Tree – deletion

Node to be deleted – Node with two children

Node to be deleted -25

- Search the node to be deleted.
- Find out inorder successor/inorder predecessor of the node.
- Copy the content into the node and delete inorder successor/in-order predecessor.



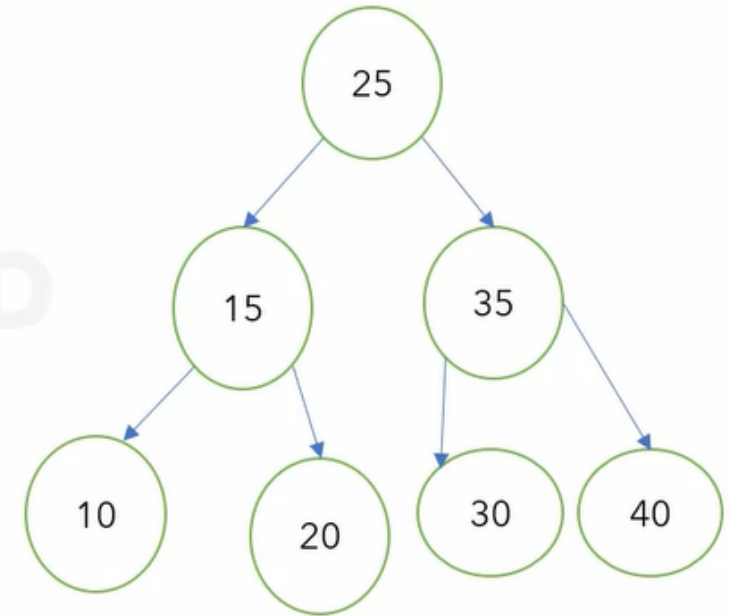
Inorder successor of 25 - node that comes after 25 is 30(smallest key in right subtree of 25)

Inorder predecessor of 25 - node that comes before 25 is 20(largest key in left subtree of 25)

Binary Search Tree – deletion

Find Inorder successor of a node with two children

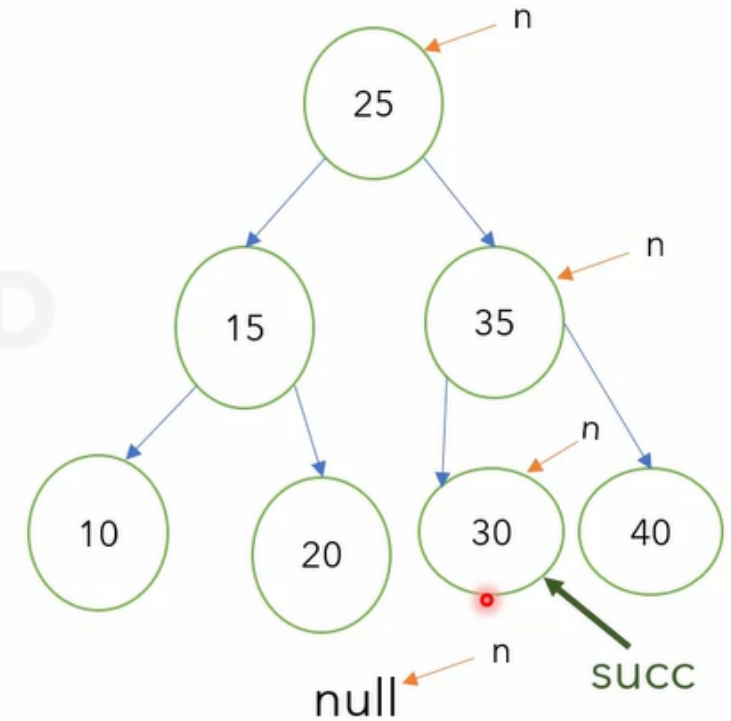
```
Inorder_Succ( TreeNode n){  
    TreeNode succ=null;  
    if(n!=null) {  
        TreeNode t=n.getRight();  
        while(t!=null) {  
            succ=t;  
            t=t.getLeft();  
        }  
        return succ;  
    }else {  
        return null;  
    } }  
}
```



Binary Search Tree – deletion

Find Inorder successor of a node with two children

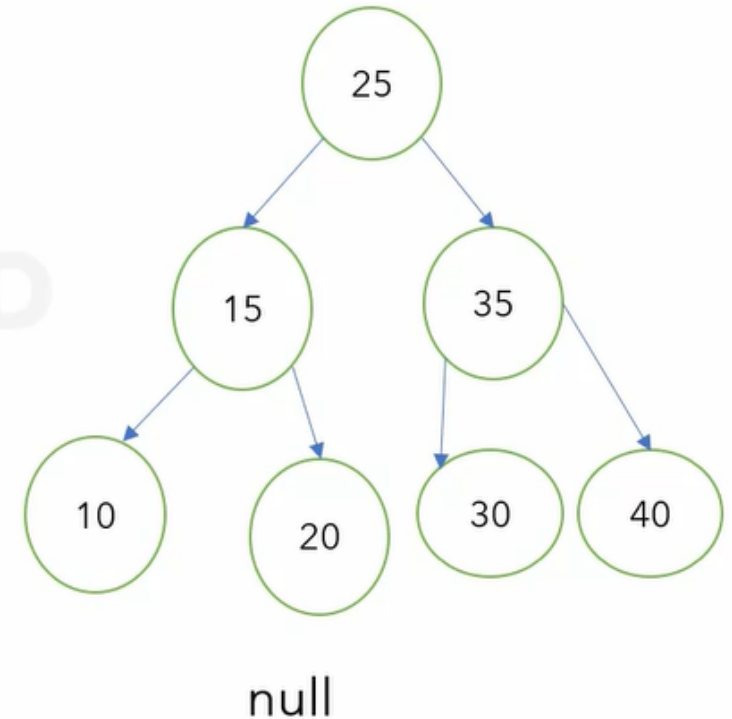
```
Inorder_Succ( TreeNode n){  
    TreeNode succ=null;  
    if(n!=null) {  
        TreeNode t=n.getRight();  
        while(t!=null) {  
            succ=t;  
            t=t.getLeft();  
        }  
        return succ;  
    }else {  
        return null;  
    } }  
}
```



Binary Search Tree – deletion

Find Inorder predecessor of a node with two children

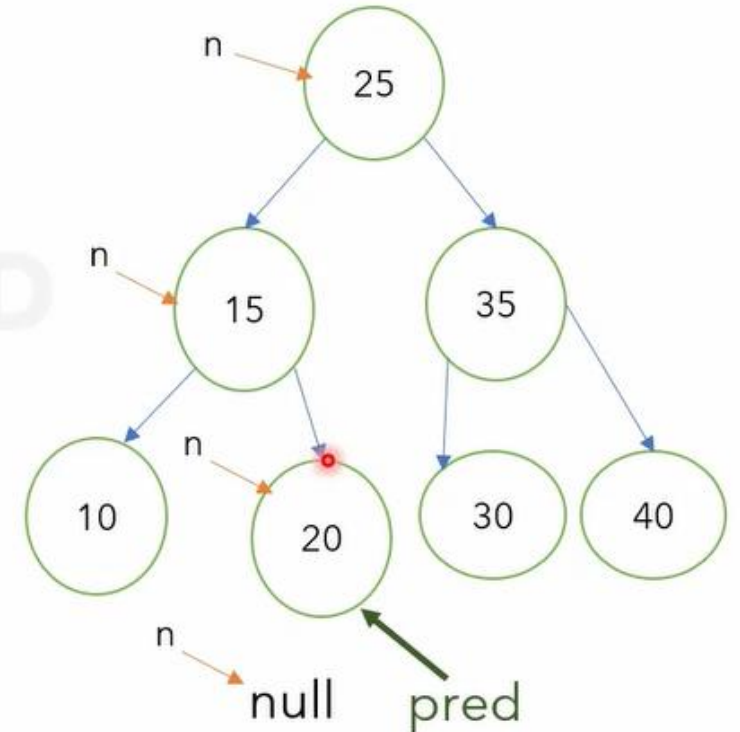
```
Inorder_pred( TreeNode n){  
    TreeNode pred=null;  
    if(n!=null){  
        TreeNode t=n.getLeft();  
        while(t!=null){  
            pred =t;  
            t=t.getRight();  
        }  
        return pred;  
    }else {  
        return null;  
    }  
}
```



Binary Search Tree – deletion

Find Inorder predecessor of a node with two children

```
Inorder_pred( TreeNode n){  
    TreeNode pred=null;  
    if(n!=null){  
        TreeNode t=n.getLeft();  
        while(t!=null){  
            pred =t;  
            t=t.getRight();  
        }  
        return pred;  
    }else {  
        return null;  
    }  
}
```



Binary Search Tree – deletion

```
TreeNode deleteNode(TreeNode r, int k) {
    TreeNode t;

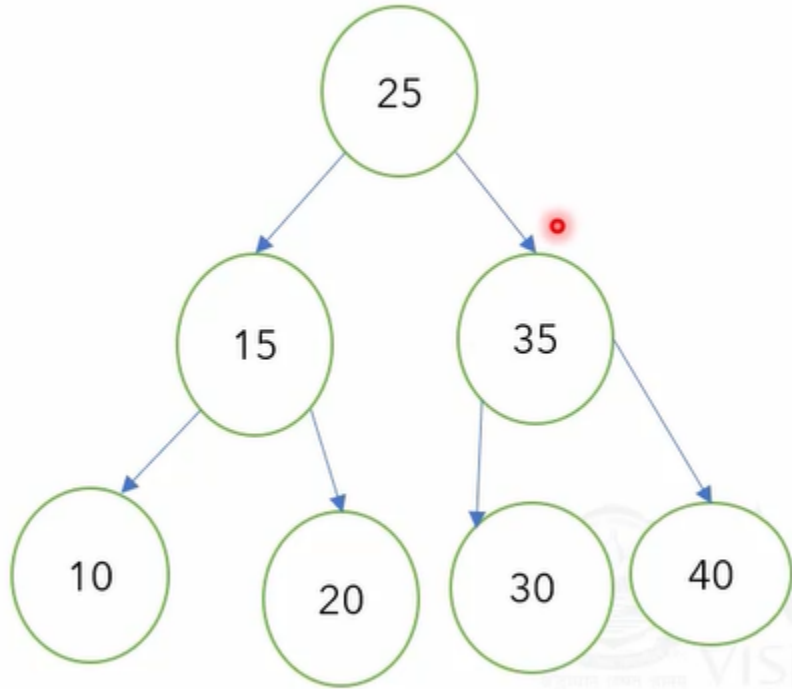
    if (r==null) return r;

    else if(k < r.getData()) {
        t=deleteNode(r.getLeft(), k);
        r.setLeft(t);
    }

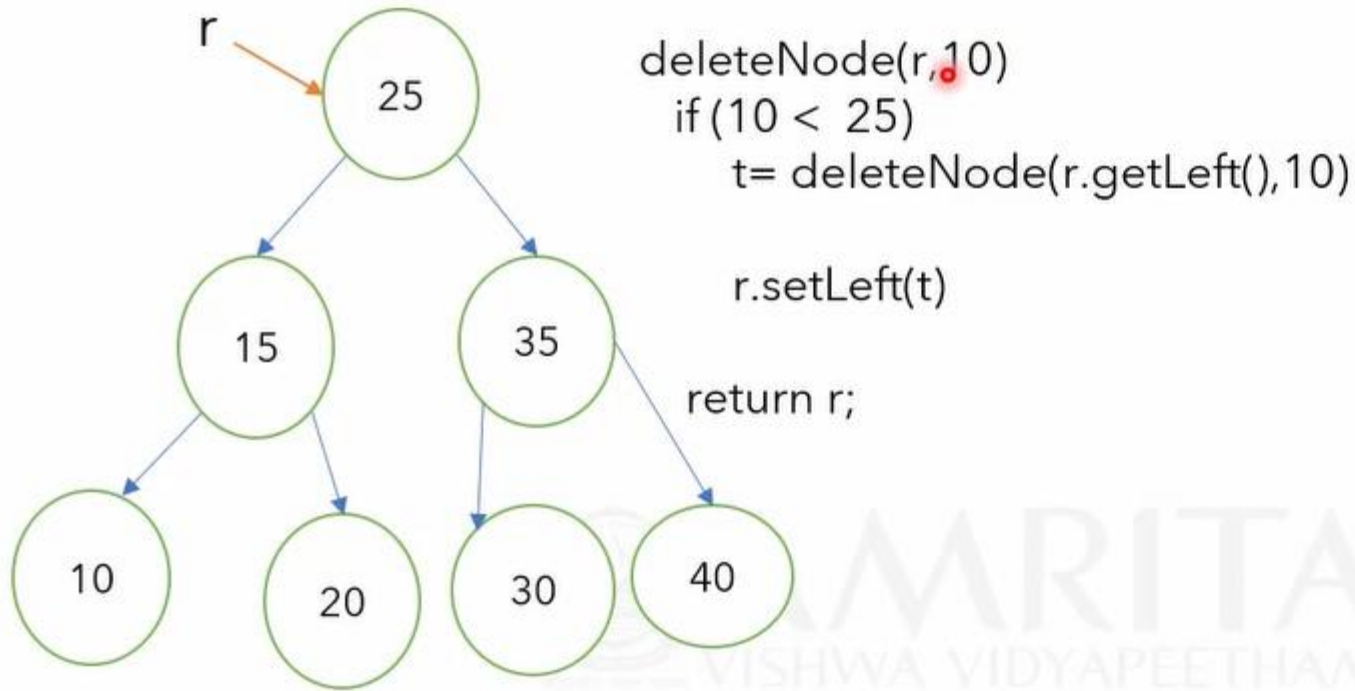
    else if(k>r.getData()) {
        t=deleteNode(r.getRight(), k);
        r.setRight(t);
    }

    else { /*node to delete*/
        if(r.getLeft()==null) {
            return r.getRight();
        }
        else if(r.getRight()==null) {
            return r.getLeft();
        }
        else {
            int v=getInorderSuccValue(r.getRight());
            r.setData(v);
            t=deleteNode(r.getRight(),v);
            r.setRight(t);
        }
        return r;
    }
}
```

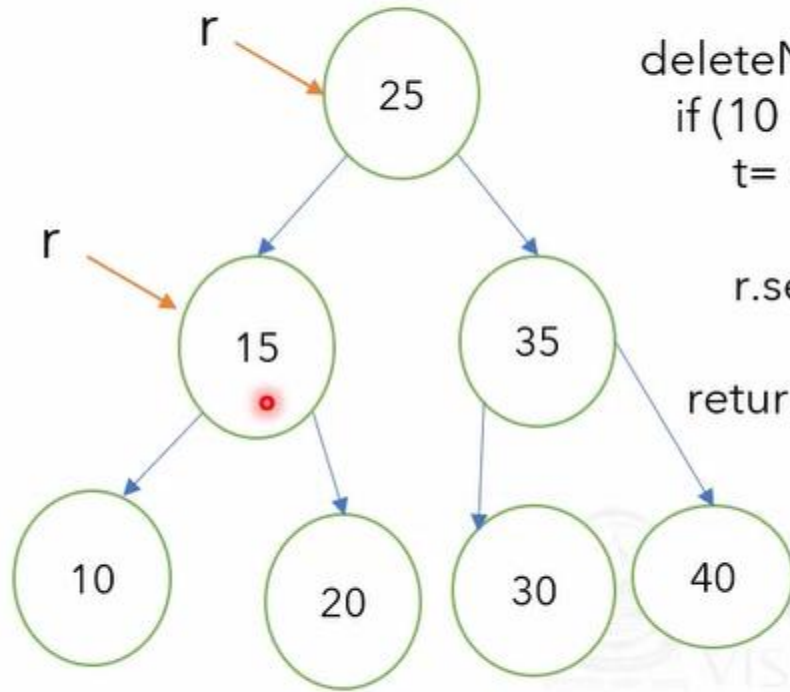
Binary Search Tree – delete 10



Binary Search Tree – delete 10



Binary Search Tree – delete 10



```
deleteNode(r,10)
```

```
if (10 < 25)
```

```
t= deleteNode(r.getLeft(),10)
```

```
r.setLeft(t)
```

```
return r;
```

```
deleteNode(r,10)
```

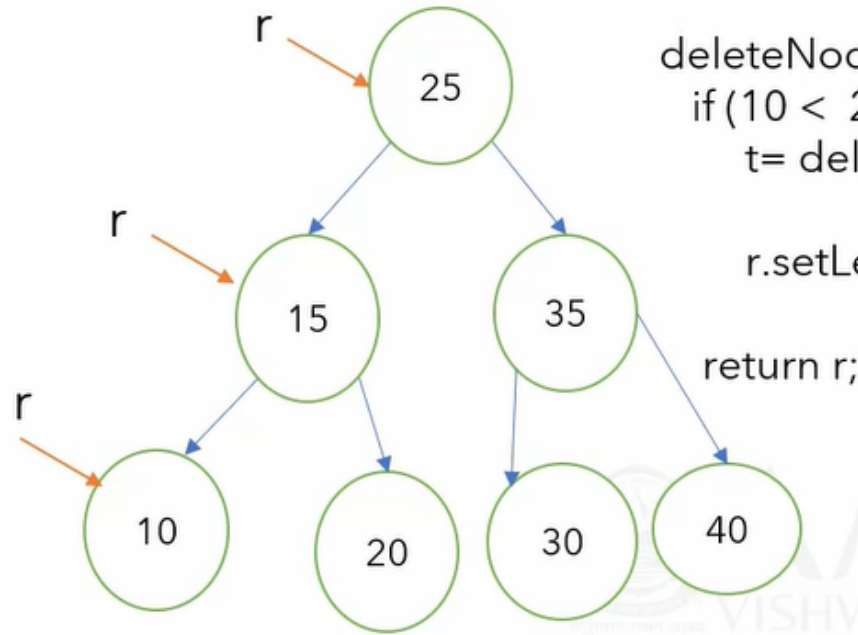
```
if (10 < 15)
```

```
t= deleteNode(r.getLeft(),10)
```

```
r.setLeft(t)
```

```
return r;
```

Binary Search Tree – delete 10



deleteNode(r,10)

if (10 < 25)

t= deleteNode(r.getLeft(),10) deleteNode(r,10)

r.setLeft(t)

return r;

if (10 < 15)

t= deleteNode(r.getLeft(),10) deleteNode(r,10)

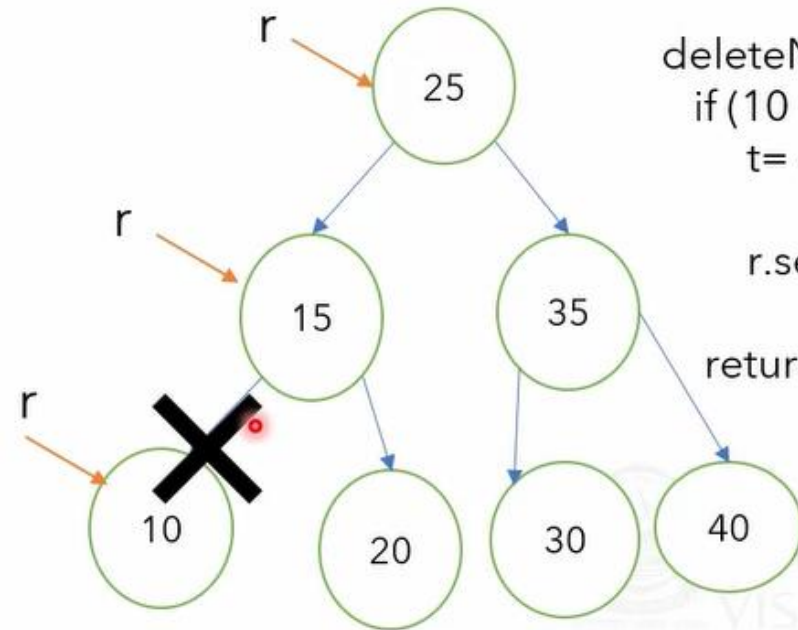
r.setLeft(t)

return r;

```

else { /*node to delete*/
if(r.getLeft()==null) {
return r.getRight();
}
.....
}
  
```

Binary Search Tree – delete 10



```
deleteNode(r,10)
```

```
if (10 < 25)
```

```
    t= deleteNode(r.getLeft(),10)
```

```
    r.setLeft(t)
```

```
    return r;
```

```
deleteNode(r,10)
```

```
if (10 < 15)
```

```
    t= deleteNode(r.getLeft(),10)
```

```
    null
```

```
    r.setLeft(t)
```

```
    return r;
```

```
deleteNode(r,10)
```

```
else { /*node to delete*/
```

```
    if(r.getLeft()==null) {
```

```
        return r.getRight();
```

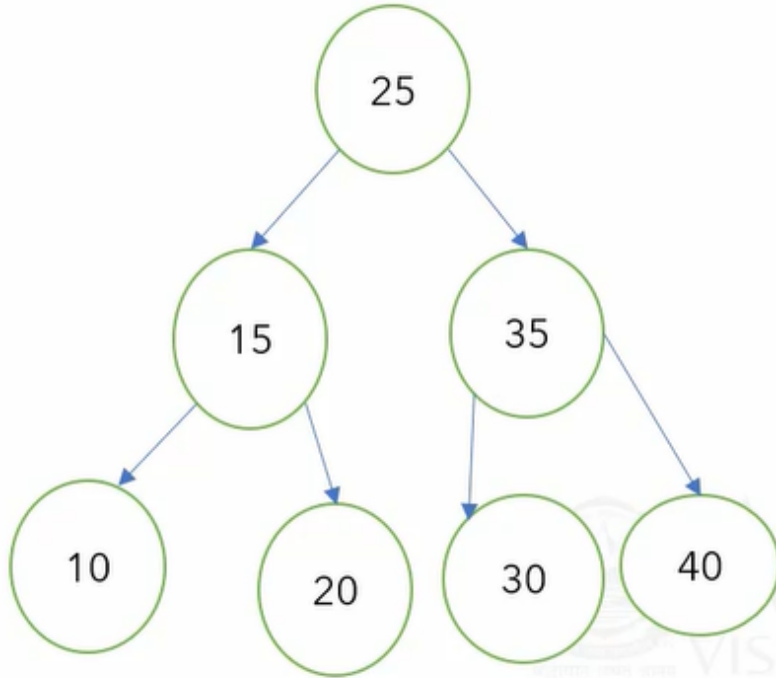
```
        null
```

```
    }
```

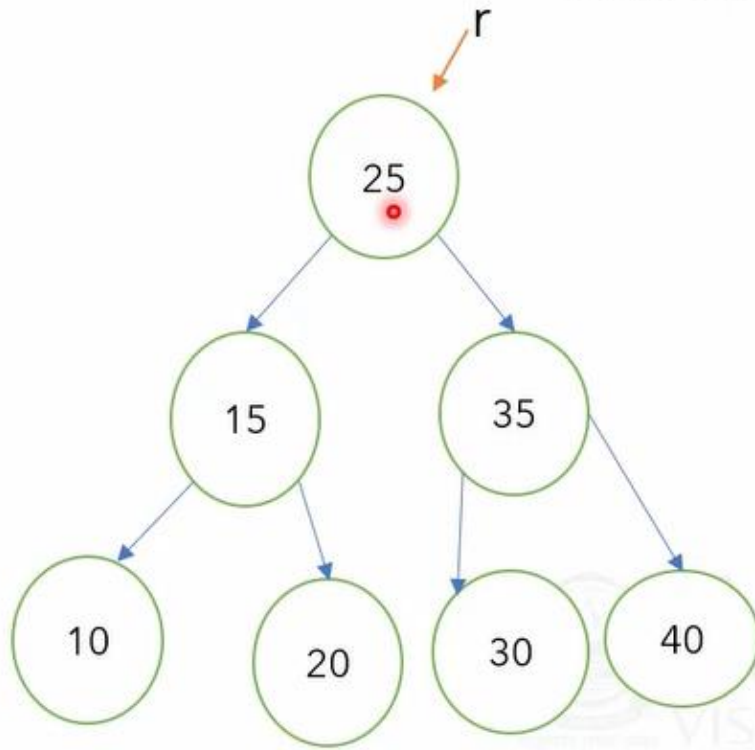
```
    .....
```

```
    }
```

Binary Search Tree – delete 25



Binary Search Tree – delete 25



```
deleteNode(r,25)
```

```
.....
```

```
else { /*node to delete*/
```

```
.....
```

```
int v=getInorderSuccValue(r.getRight());
```

```
r.setData(v);
```

```
t=deleteNode(r.getRight(),v);
```

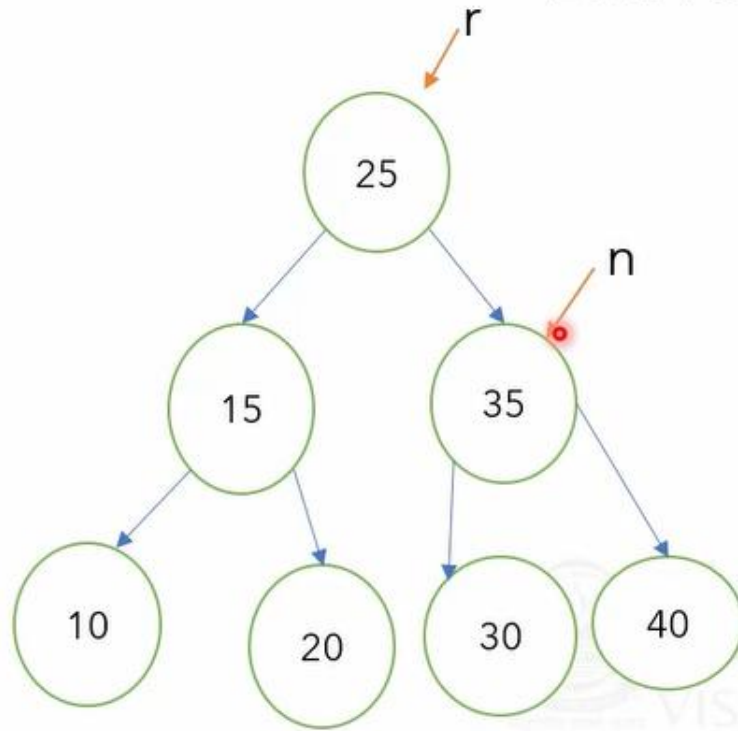
```
r.setRight(t);
```

```
}
```

```
return r;
```

```
}
```

Binary Search Tree – delete 25



```
deleteNode(r,25)
```

```
.....
```

```
else { /*node to delete*/
```

```
.....
```

```
int v=getInorderSuccValue(r.getRight());
```

```
r.setData(v);
```

```
t=deleteNode(r.getRight(),v);
```

```
r.setRight(t);
```

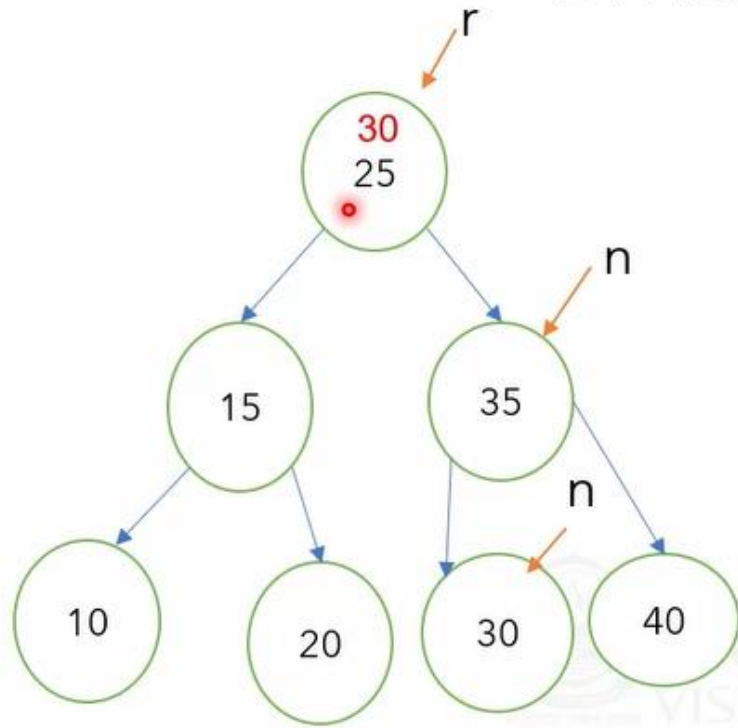
```
}
```

```
return r;
```

```
}
```

```
getInorderSuccValue(n){  
    while(n.getLeft()!=null)  
        n=n.getLeft();  
    return n.getData();  
}
```

Binary Search Tree – delete 25



```
deleteNode(r,25)
```

```
.....
```

```
else { /*node to delete*/
```

```
.....
```

```
int v=getInorderSuccValue(r.getRight());
```

```
30
```

```
r.setData(v);
```

```
t=deleteNode(r.getRight(),v);
```

```
r.setRight(t);
```

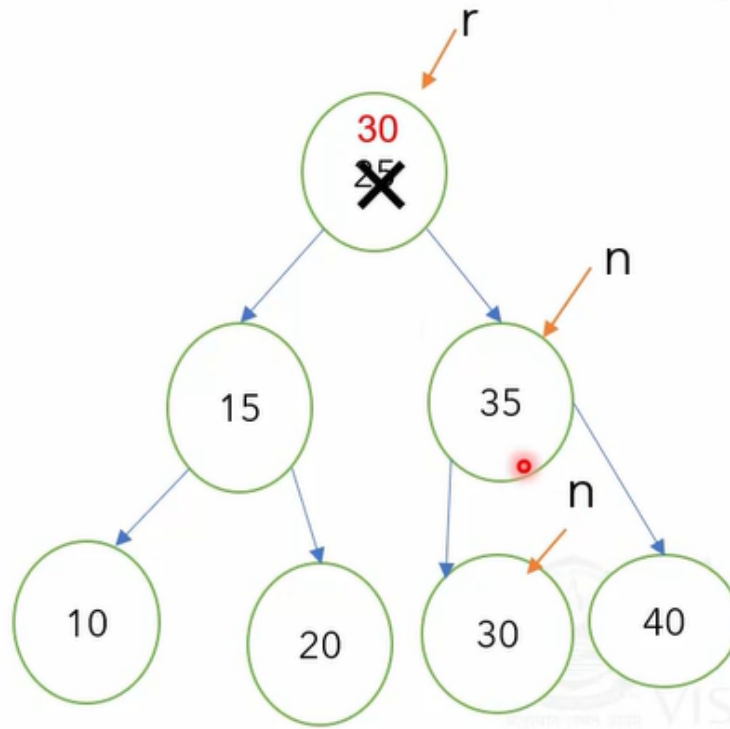
```
}
```

```
return r;
```

```
}
```

```
getInorderSuccValue(n) {
    while(n.getLeft()!=null)
        n=n.getLeft();
    return n.getData(); 30
}
```

Binary Search Tree – delete 25



```
deleteNode(r,25)
```

```
.....
```

```
else { /*node to delete*/
```

```
.....
```

```
int v=getInorderSuccValue(r.getRight());
```

```
30
```

```
r.setData(v);
```

```
t=deleteNode(r.getRight(),v);
```

```
r.setRight(t);
```

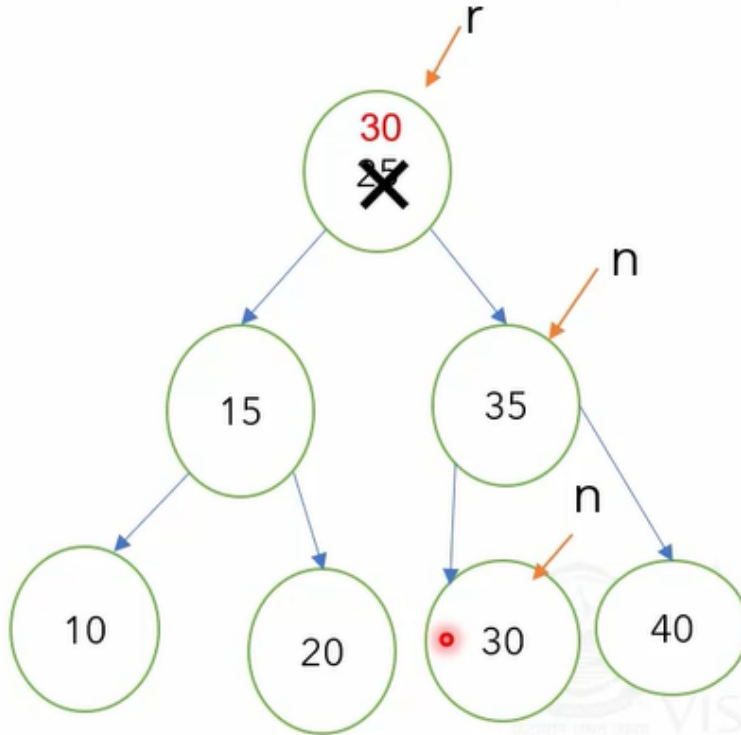
```
}
```

```
return r;
```

```
}
```

```
getInorderSuccValue(n){
    while(n.getLeft()!=null)
        n=n.getLeft();
    return n.getData(); 30
}
```

Binary Search Tree – delete 25



```
deleteNode(r,25)
```

```
.....
```

```
else { /*node to delete*/
```

```
.....
```

```
int v=getInorderSuccValue(r.getRight());
```

```
30
```

```
r.setData(v);
```

```
t=deleteNode(r.getRight(),v);
```

```
r.setRight(t);
```

```
}
```

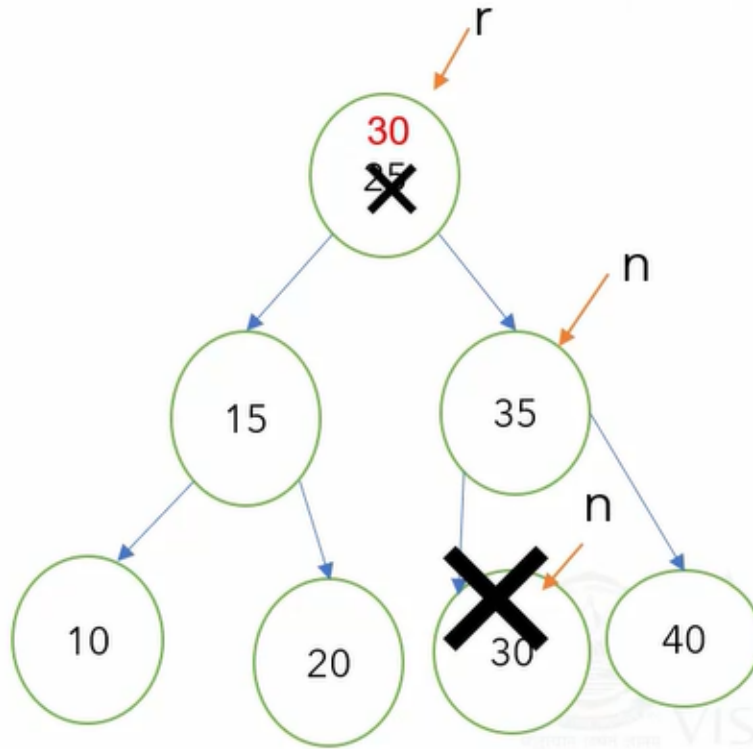
```
return r;
```

```
}
```

```
getInorderSuccValue(n){
    while(n.getLeft()!=null)
        n=n.getLeft();
    return n.getData(); 30
}
```

Deleting
leaf node
case

Binary Search Tree – delete 25



```
deleteNode(r,25)
```

```
.....
```

```
else { /*node to delete*/
```

```
.....
```

```
int v=getInorderSuccValue(r.getRight());
```

```
30
```

```
r.setData(v);
```

```
t=deleteNode(r.getRight(),v);
```

```
r.setRight(t);
```

```
}
```

```
return r;
```

```
}
```

```
getInorderSuccValue(n) {  
    while(n.getLeft()!=null)  
        n=n.getLeft();  
    return n.getData(); 30  
}
```

Deleting
leaf node
case

Summary

Binary search Tree

Deletion





AHEAD Online

Data Structures and Algorithms

Geetha Lekshmy V

Harsha Raj

Department of Computer Science

Amrita Vishwa Vidyapeetham

Objectives

AVL Tree

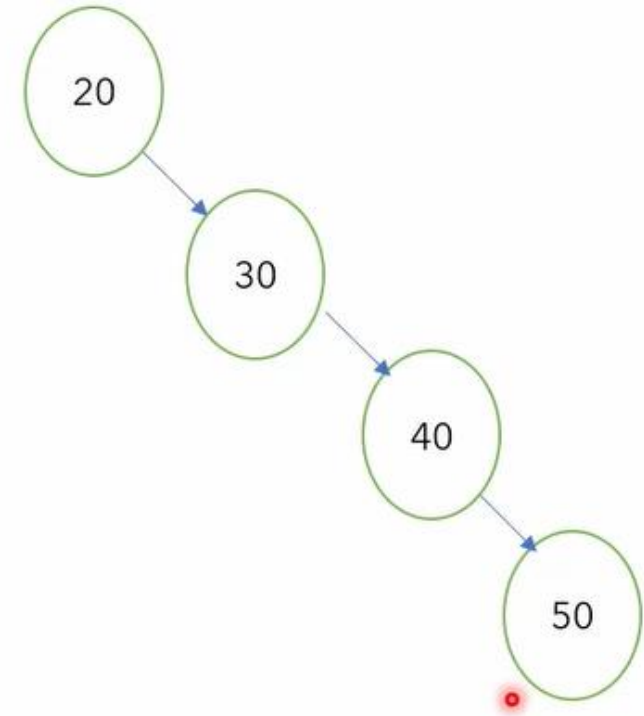
AVL Tree Operations



Place your
Webcam Video here
Size 100%

Binary Search Tree - Search

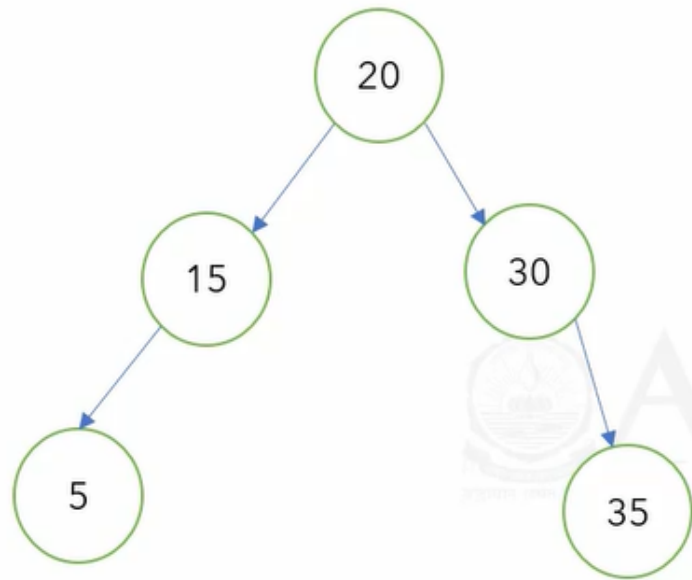
- To search for a key 70 in the given tree how many comparisons should be made?
- Time complexity of searching an element in a complete binary search tree - $O(\log_2 n)$
- Time complexity of searching an element in a binary search tree is $O(n)$, in the worst case.



AVL(Adelson-Velskii and Landis) Tree

- Binary Search Tree
- Height Balanced Tree – The height of the left subtree and right subtree differ by at most one.
- Due to this height-balance property, the height of an AVL tree storing 'n' entries is $O(\log n)$.

AVL Tree



Balance factor of Node 20 = height of left subtree - height of right subtree.

$$= 2 - 2 = 0$$

Balance factor of Node 15 = 1 - 0 = 1

Balance factor of Node 30 = 0 - 1 = -1

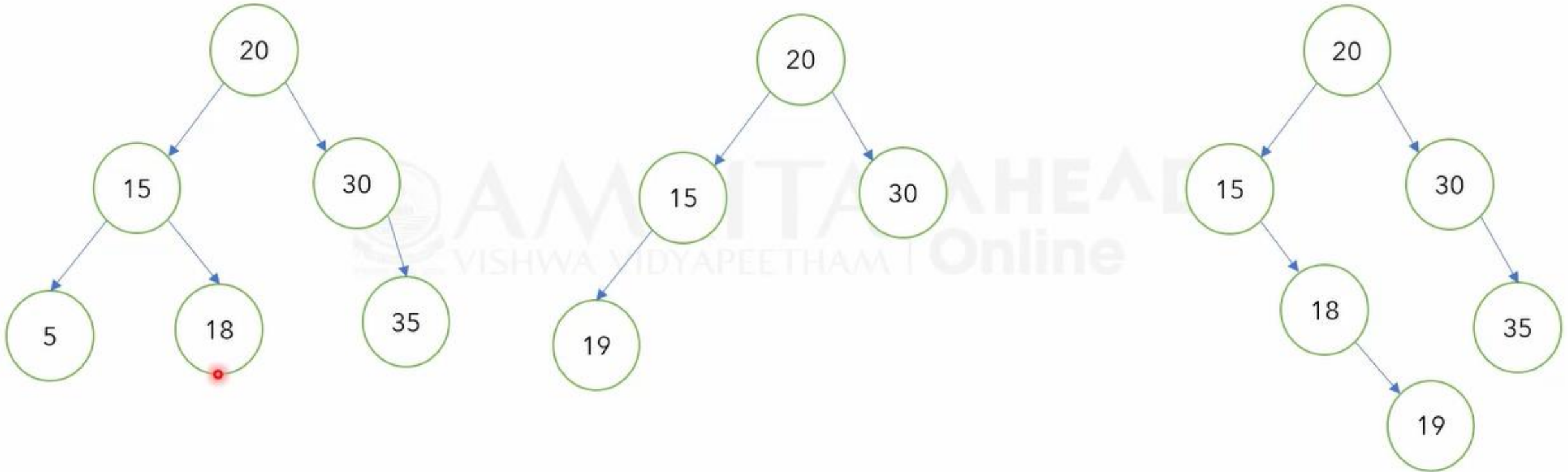
Balance factor of Node 5, 35 = 0 - 0 = 0

Each node of the tree satisfies height - balance property.

Balance Factor can have the values -0, 1, -1

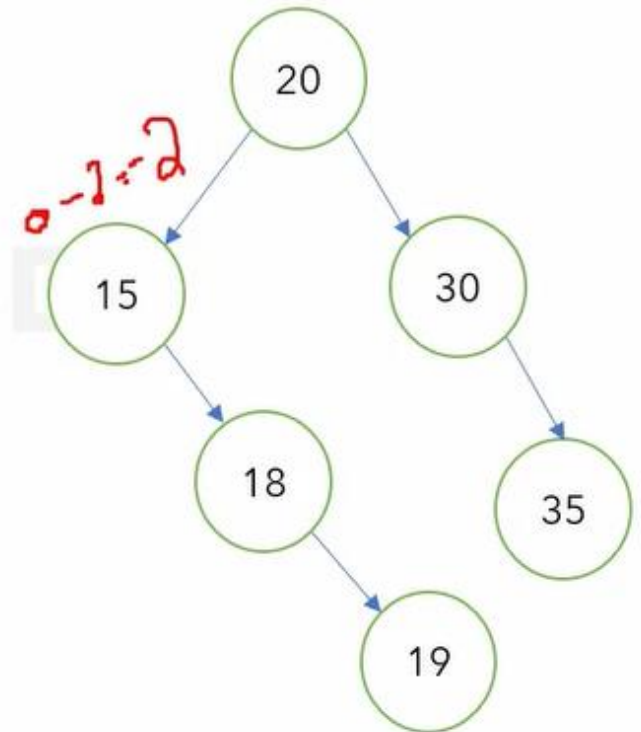
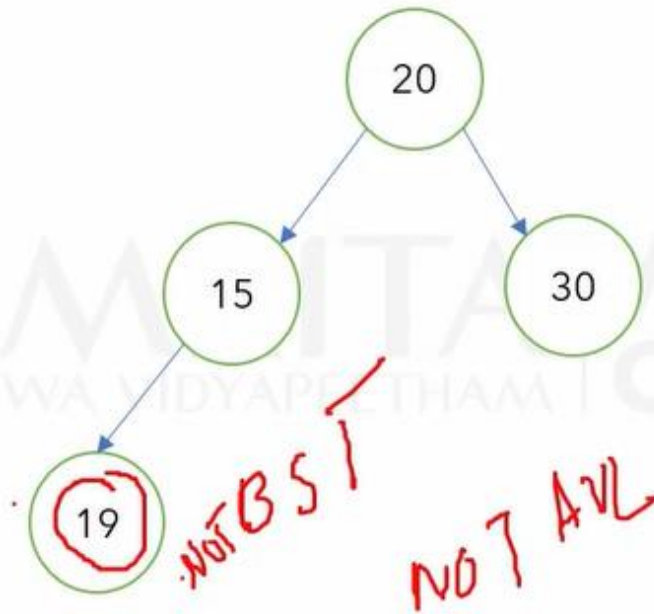
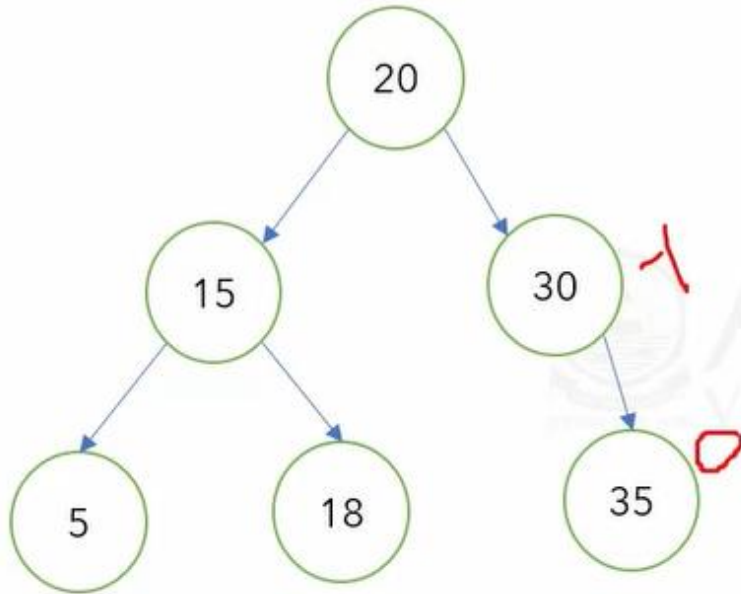
AVL Tree

Which of the trees are AVL?



AVL Tree

Which of the trees are AVL?

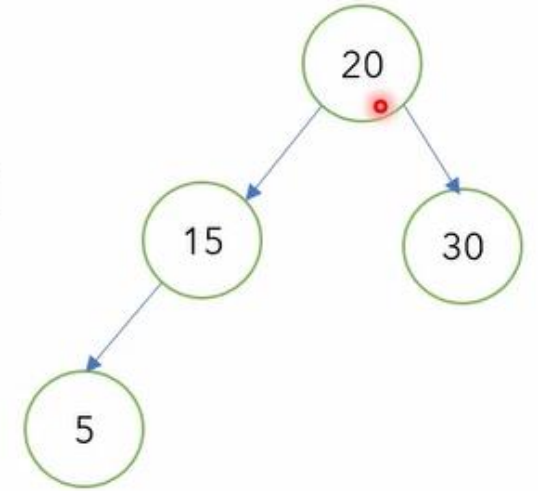


AVL Tree - Insertion

Case-1

Single right rotation

Inserting 3 - node 15 becomes unbalanced, perform single right rotation at node 15.

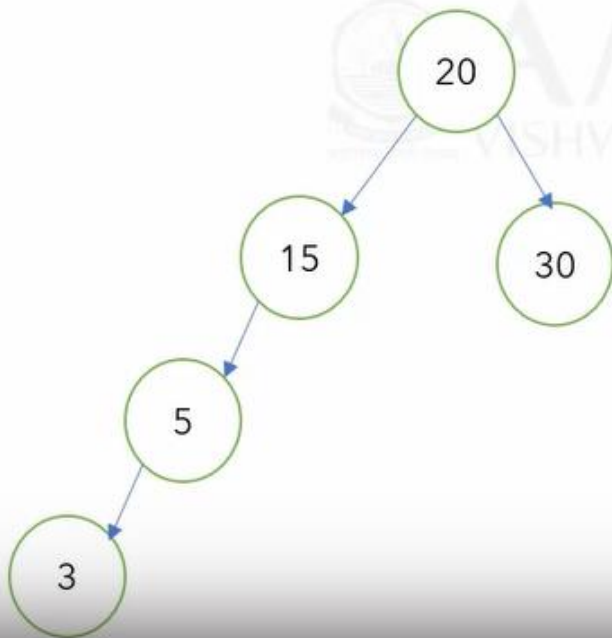
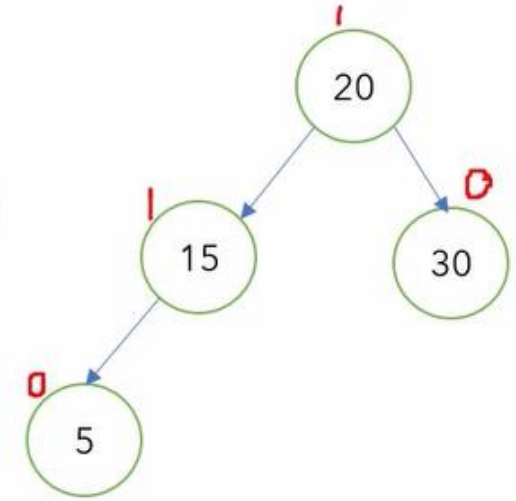


AVL Tree - Insertion

Case-1

Single right rotation

Inserting 3 - node 15 becomes unbalanced, perform single right rotation at node 15.

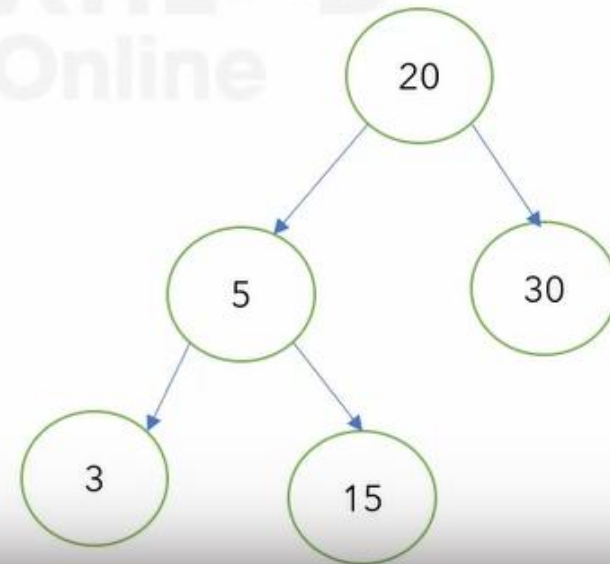
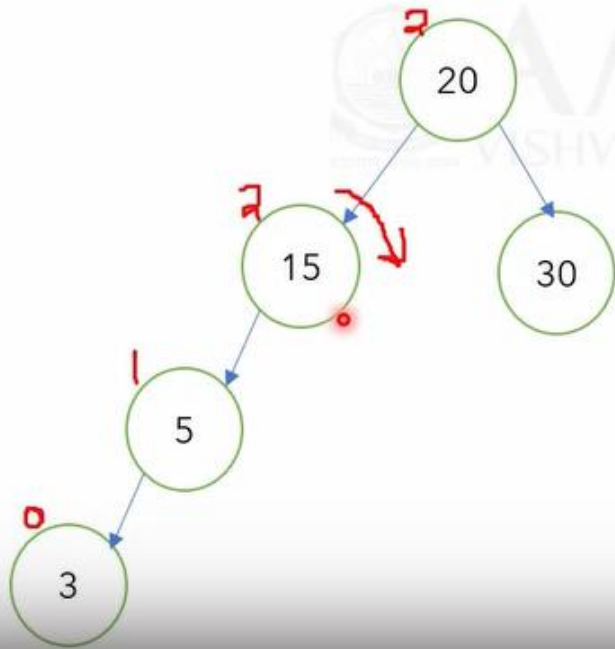
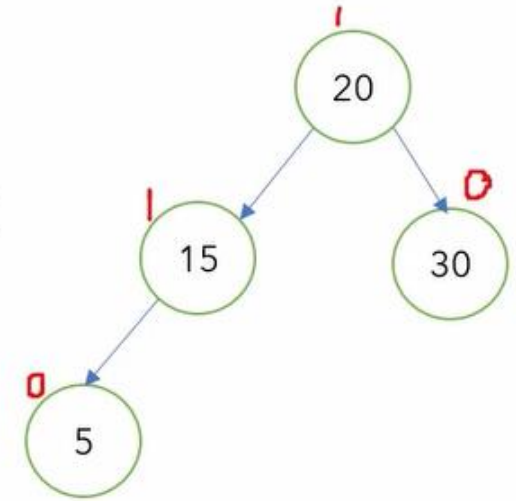


AVL Tree - Insertion

Case-1

Single right rotation

Inserting 3 - node 15 becomes unbalanced, perform single right rotation at node 15.

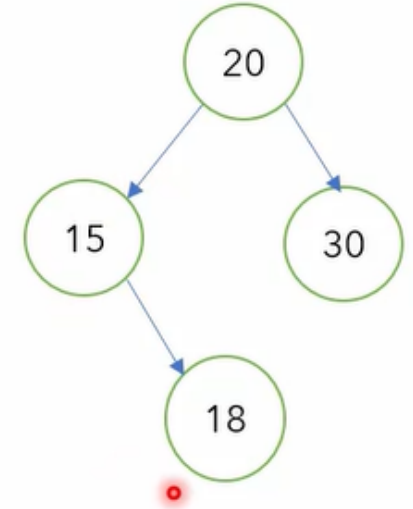


AVL Tree - Insertion

Case-2

Single left rotation

Inserting 19 – node 15 becomes unbalanced, perform single left rotation at node 15.

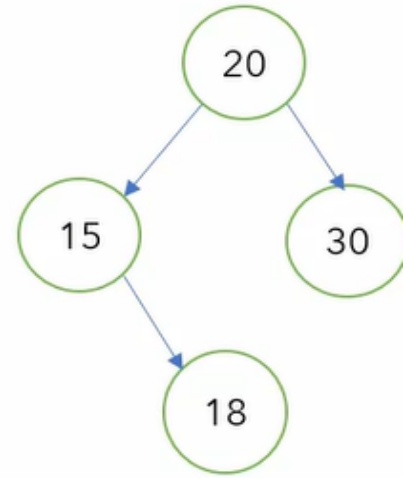


AVL Tree - Insertion

Case-2

Single left rotation

Inserting 19 - node 15 becomes unbalanced, perform single left rotation at node 15.

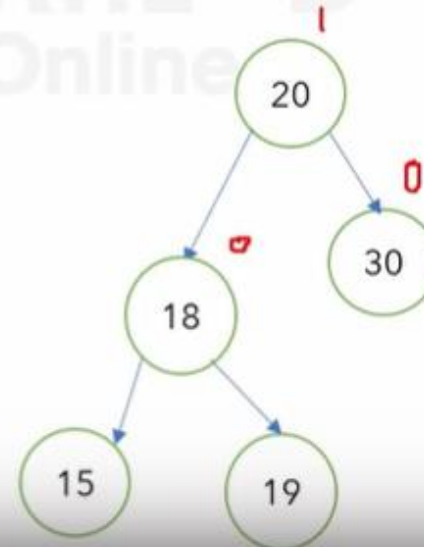
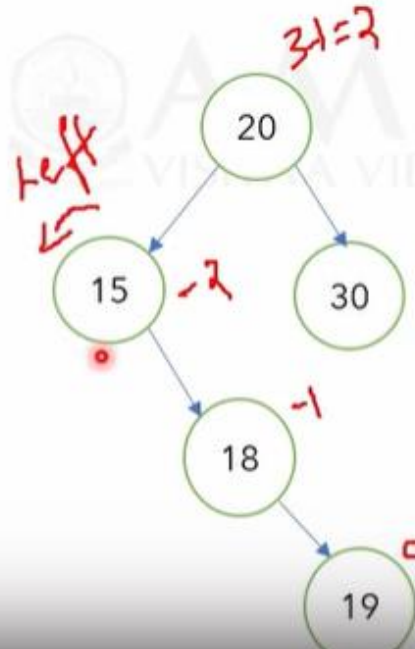
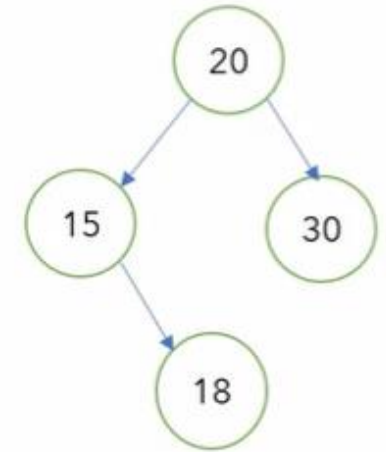


AVL Tree - Insertion

Case-2

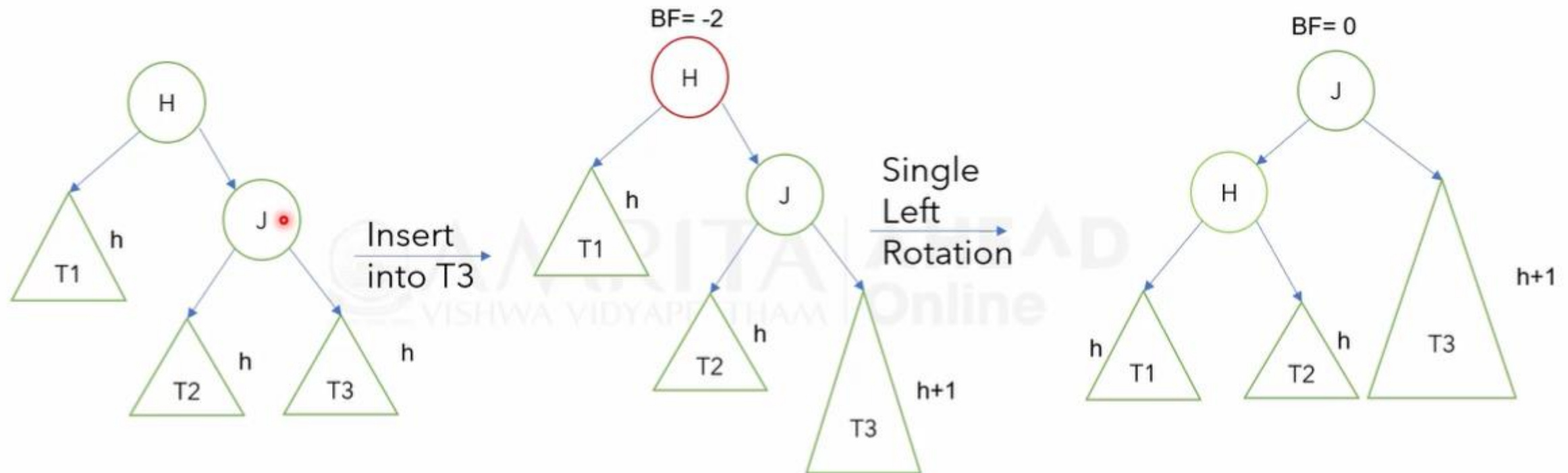
Single left rotation

Inserting 19 - node 15 becomes unbalanced, perform single left rotation at node 15.



AVL Tree - Insertion

Case-2 Single left rotation

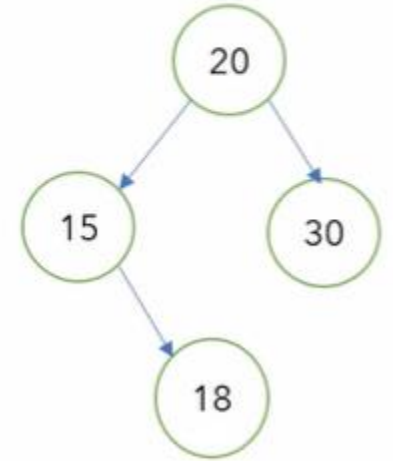


AVL Tree - Insertion

Case-3

Right-Left rotation

Inserting 16 - node 15 becomes unbalanced, perform right rotation at node 18 and the left rotation at node 15.

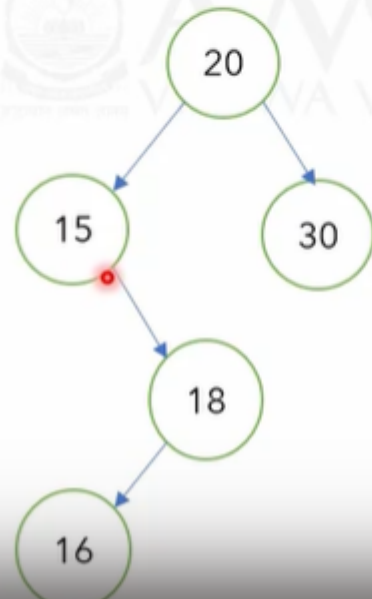
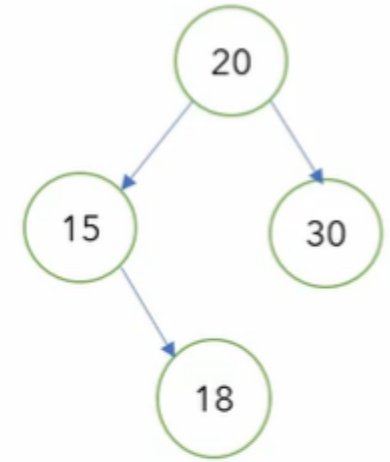


AVL Tree - Insertion

Case-3

Right-Left rotation

Inserting 16 - node 15 becomes unbalanced, perform right rotation at node 18 and the left rotation at node 15.

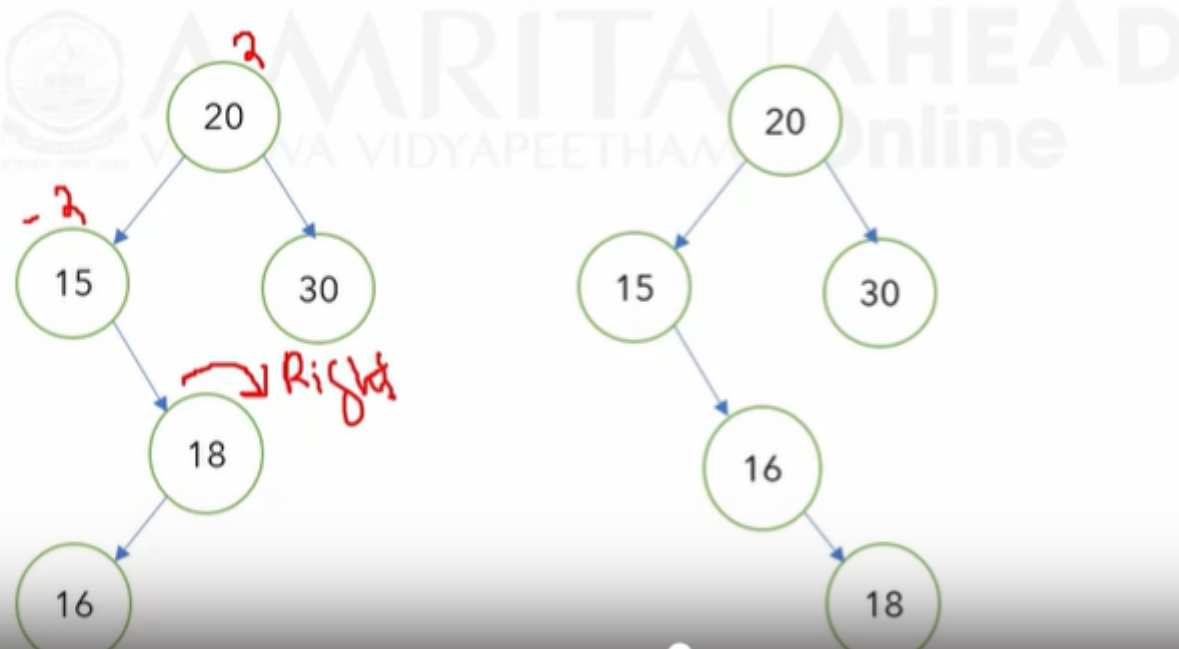
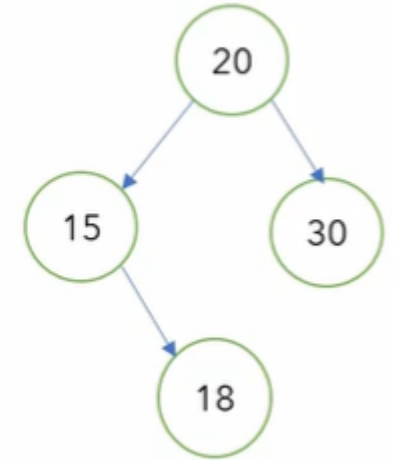


AVL Tree - Insertion

Case-3

Right-Left rotation

Inserting 16 - node 15 becomes unbalanced, perform right rotation at node 18 and the left rotation at node 15.

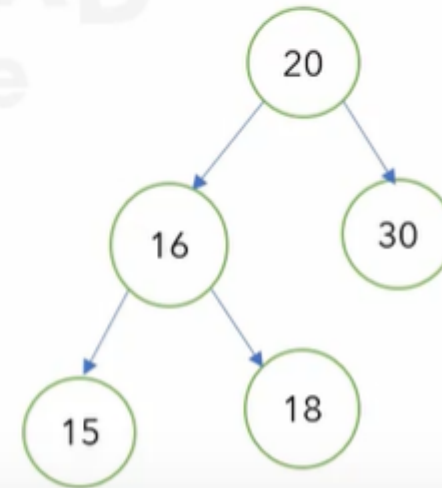
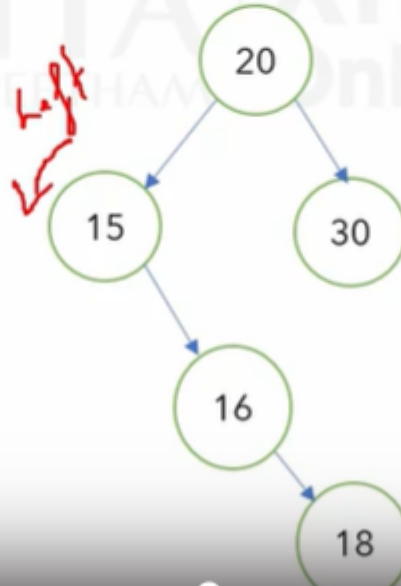
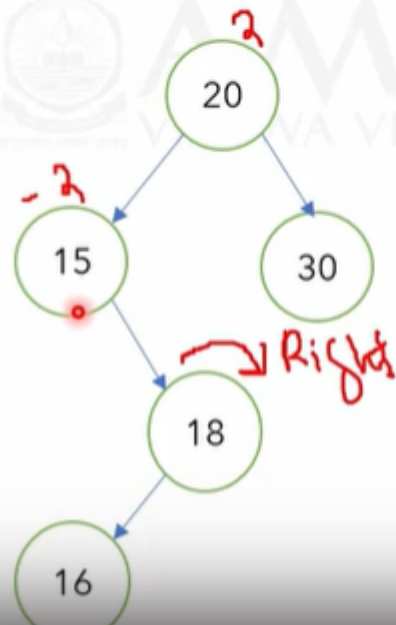
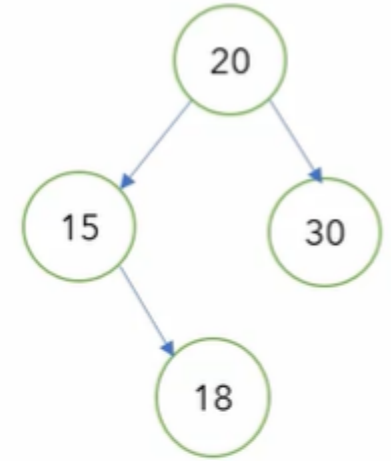


AVL Tree - Insertion

Case-3

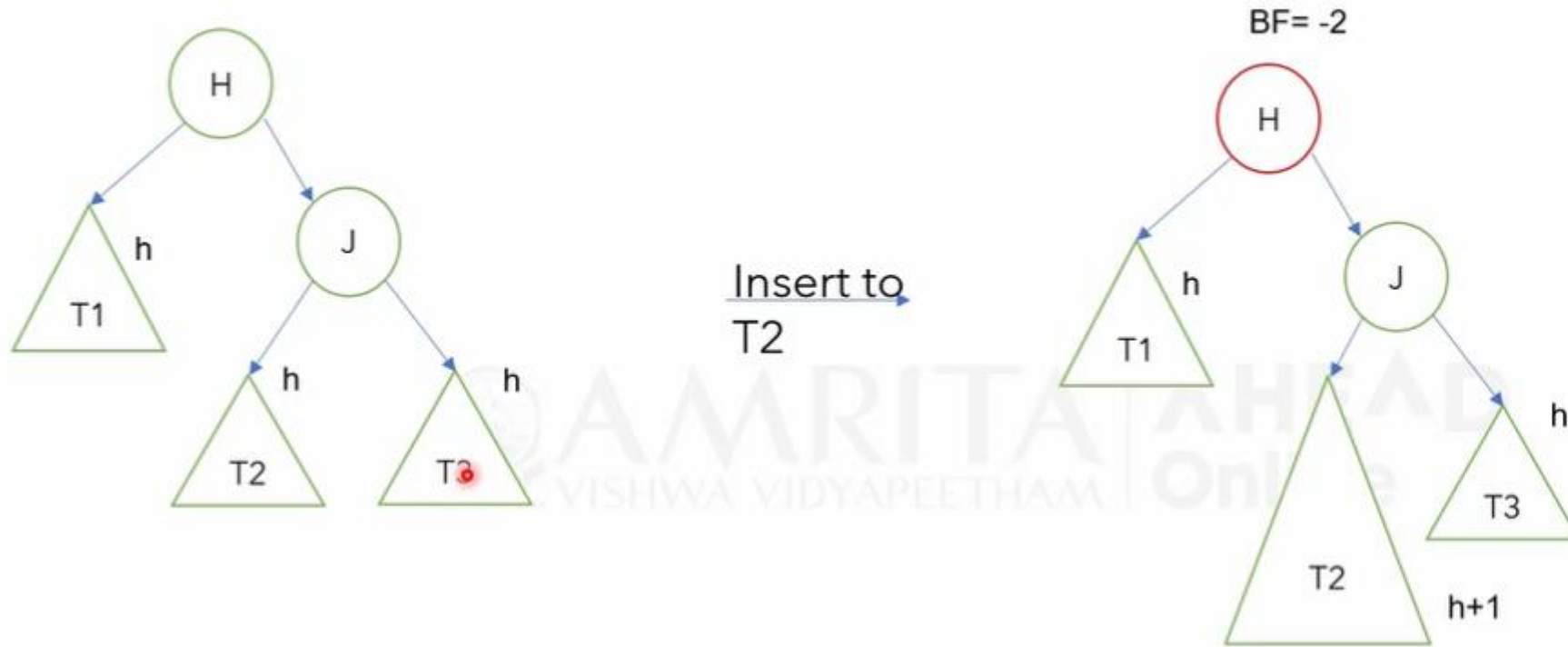
Right-Left rotation

Inserting 16 - node 15 becomes unbalanced, perform right rotation at node 18 and the left rotation at node 15.



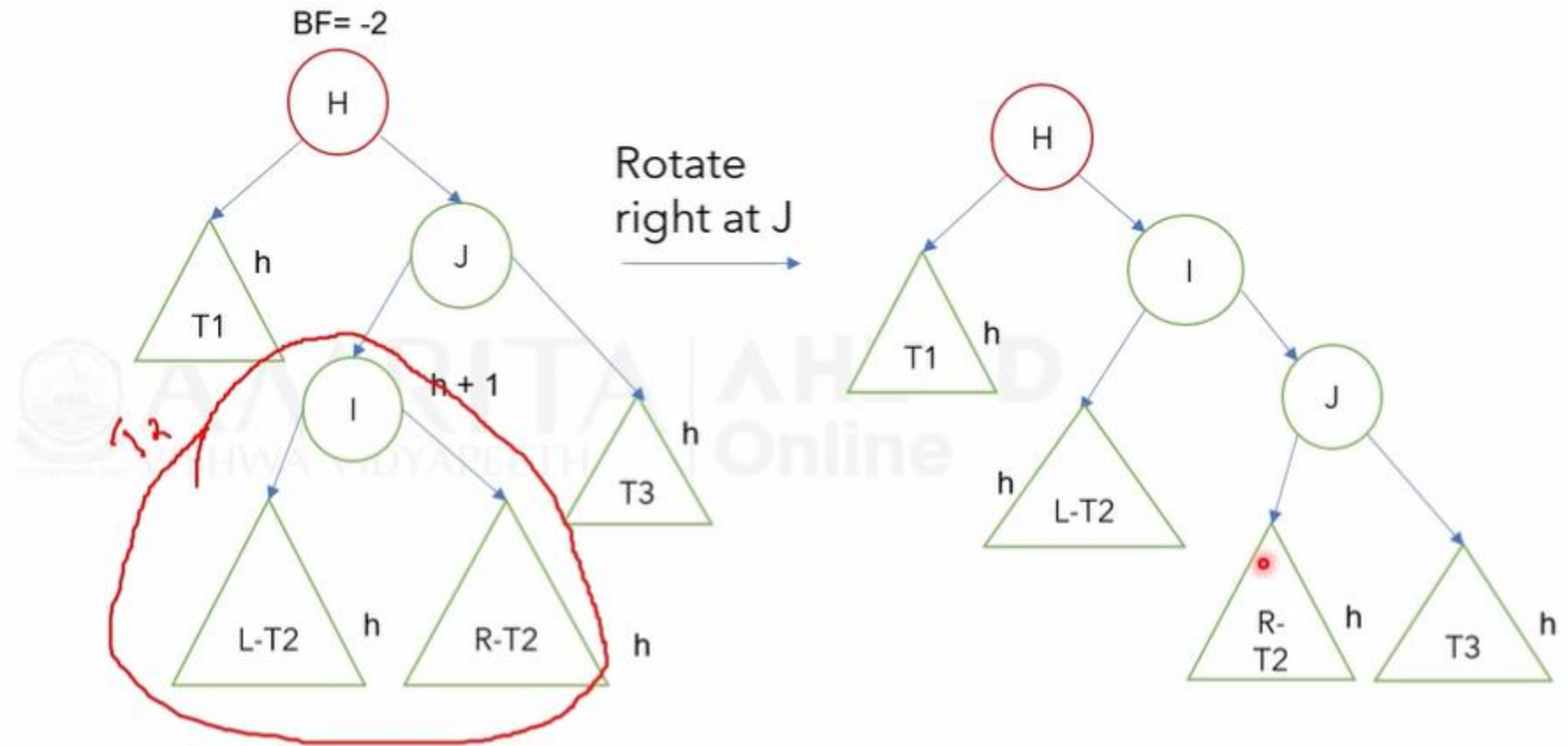
AVL Tree - Insertion

Case-3 Right-Left rotation



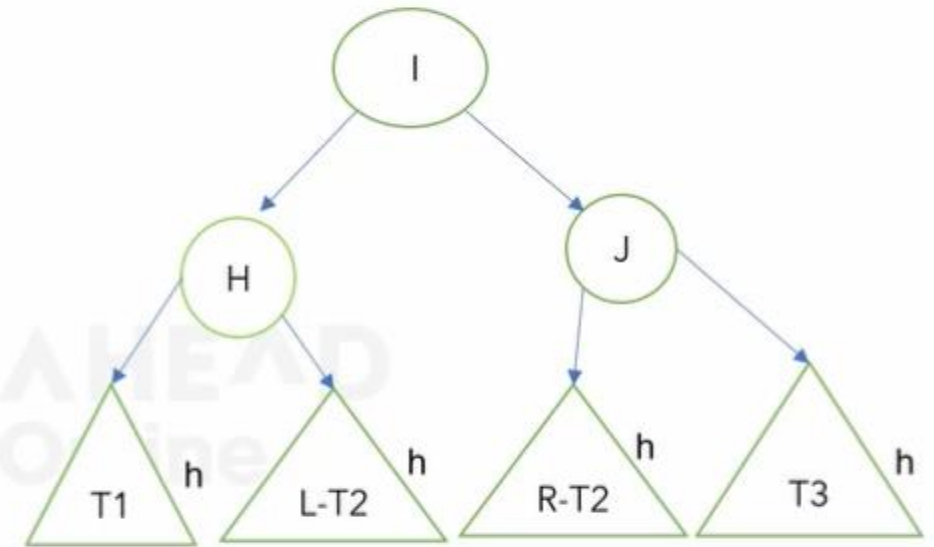
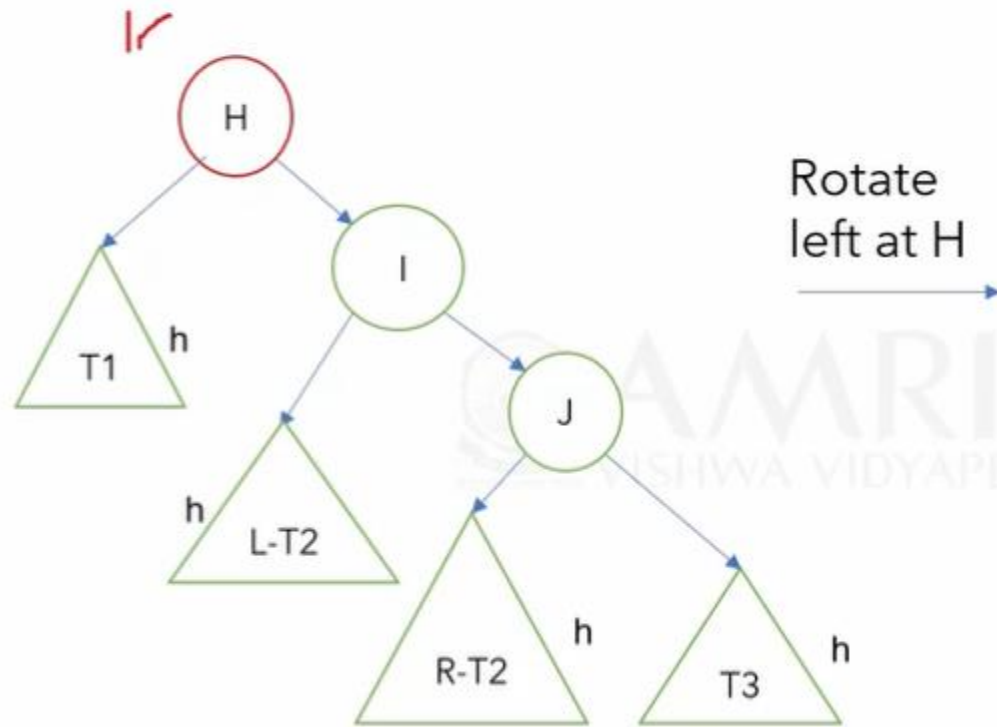
AVL Tree - Insertion

Case-3 Right-Left rotation



AVL Tree - Insertion

Case-3 Right-Left rotation

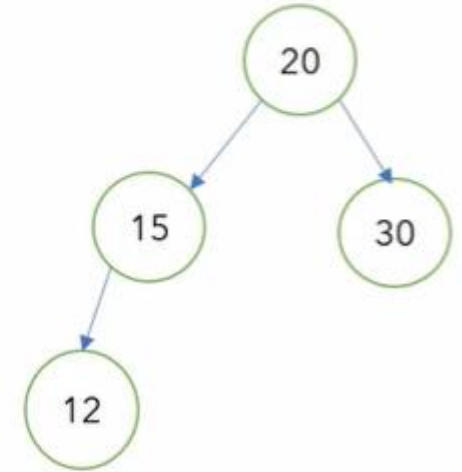


AVL Tree - Insertion

Case-4

Left-Right rotation

Inserting 13 - node 15 becomes unbalanced first, perform left rotation at node 12 and right rotation at node 15.

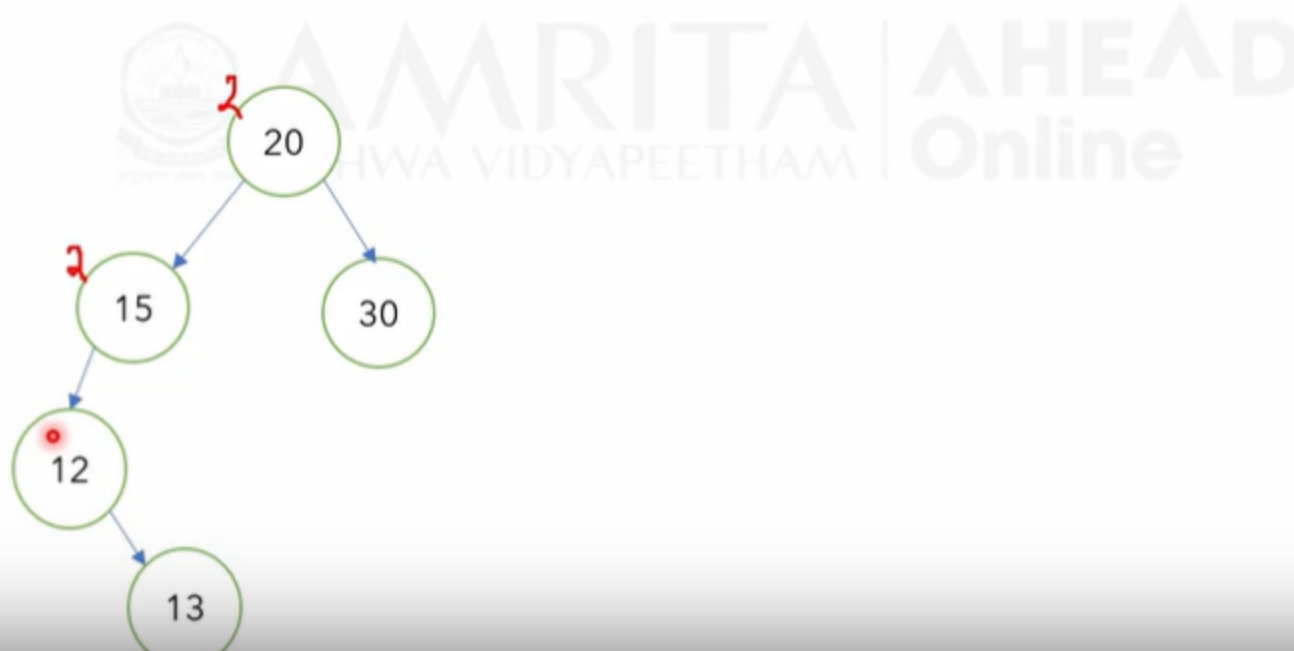
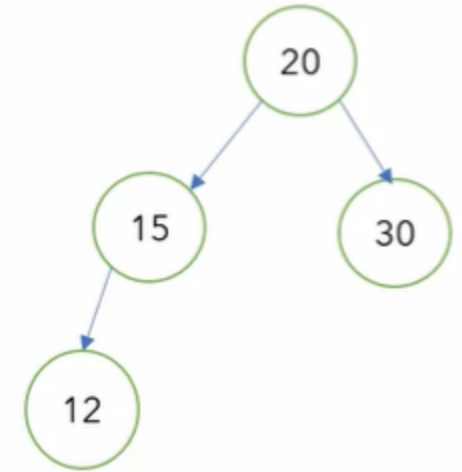


AVL Tree - Insertion

Case-4

Left-Right rotation

Inserting 13 - node 15 becomes unbalanced first, perform left rotation at node 12 and right rotation at node 15.

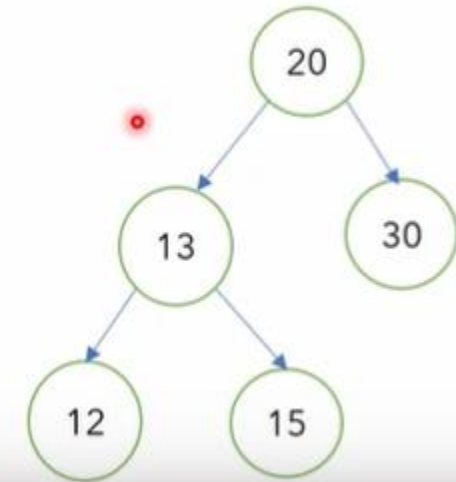
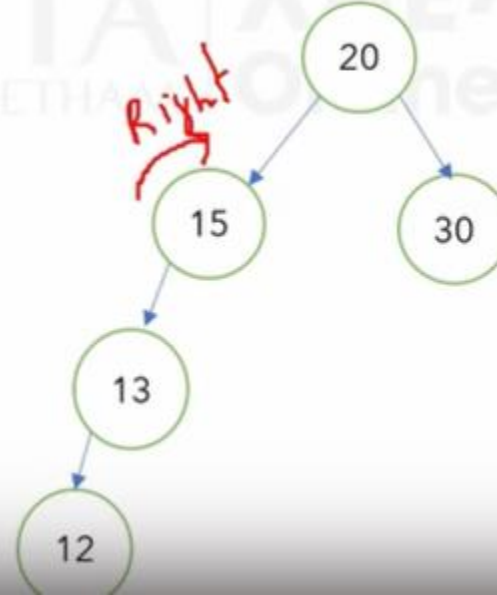
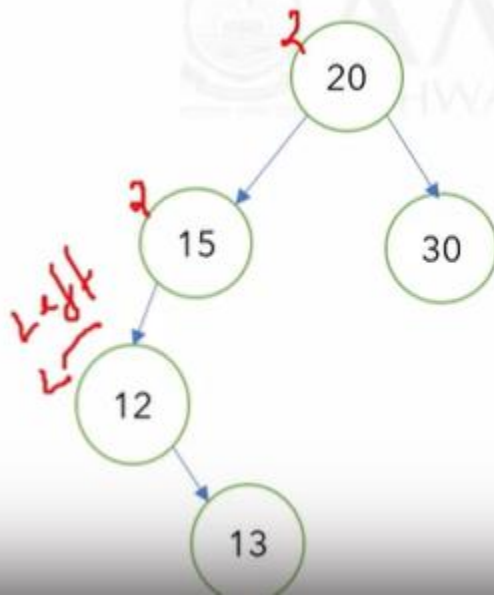
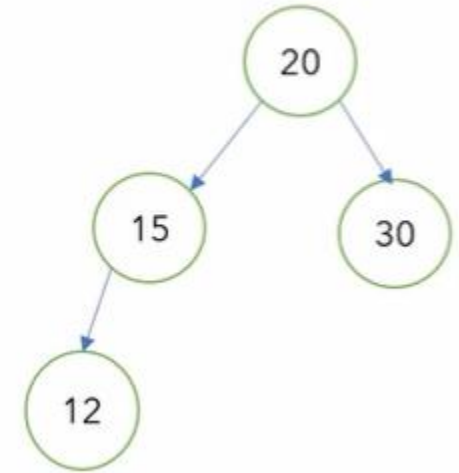


AVL Tree - Insertion

Case-4

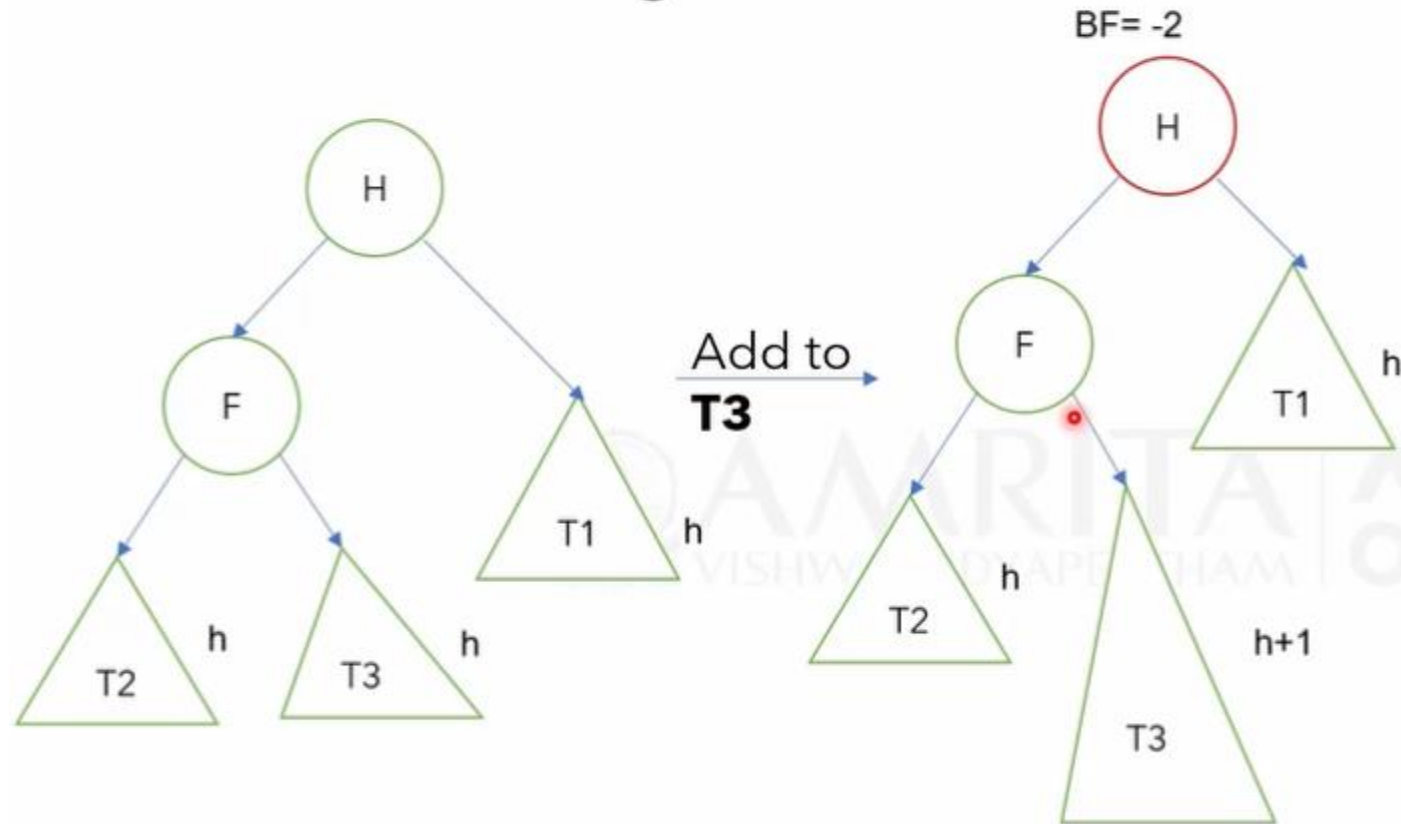
Left-Right rotation

Inserting 13 - node 15 becomes unbalanced first, perform left rotation at node 12 and right rotation at node 15.



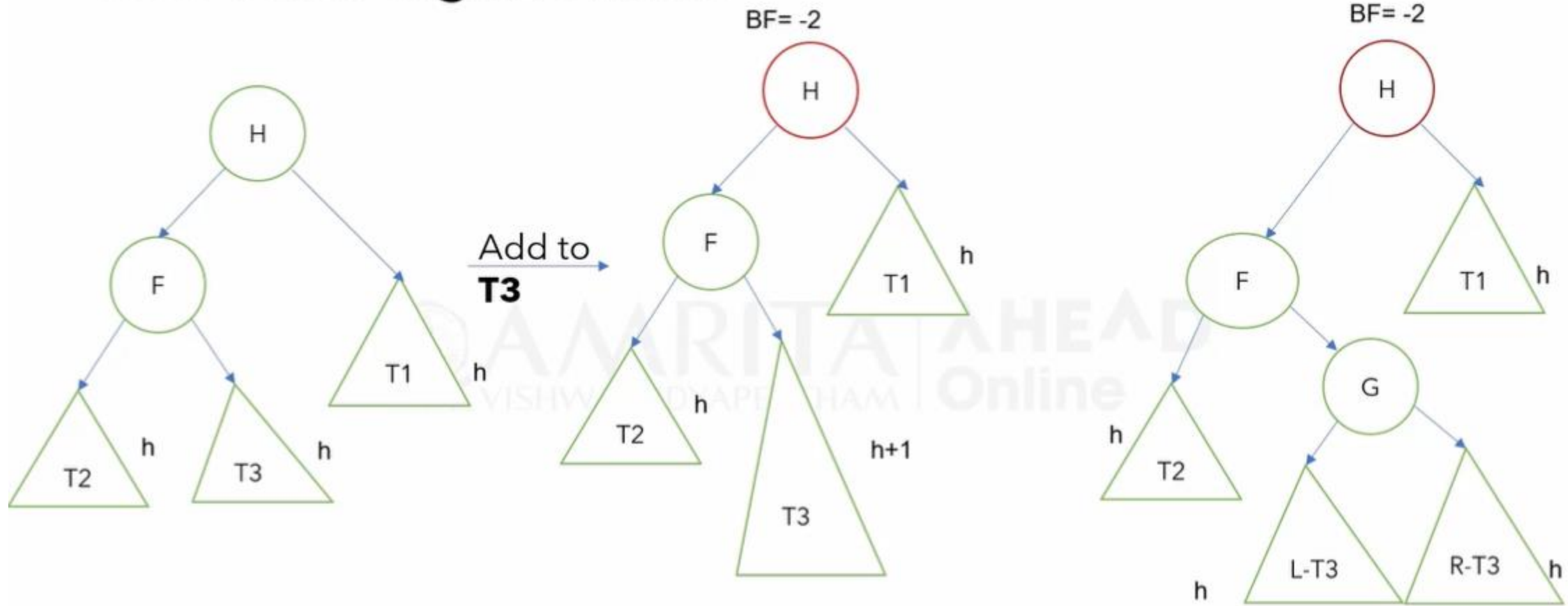
AVL Tree - Insertion

Case-4 Left -Right rotation



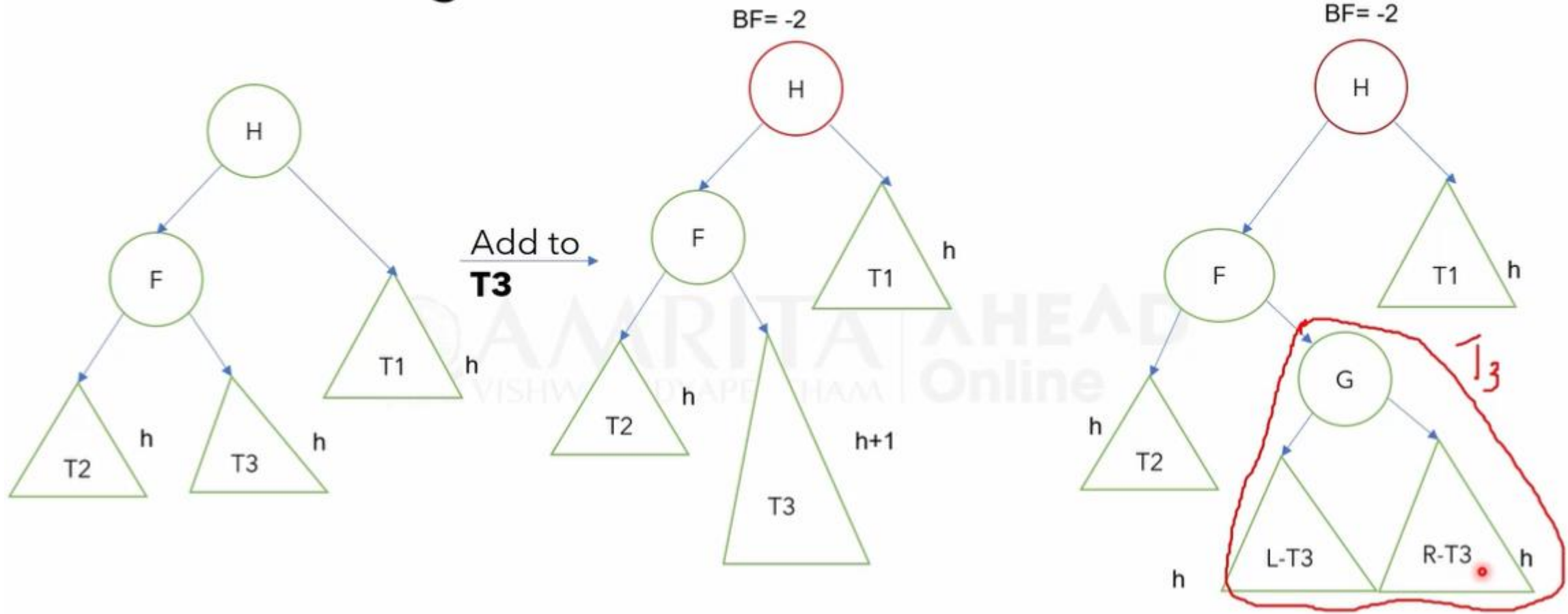
AVL Tree - Insertion

Case-4 Left -Right rotation



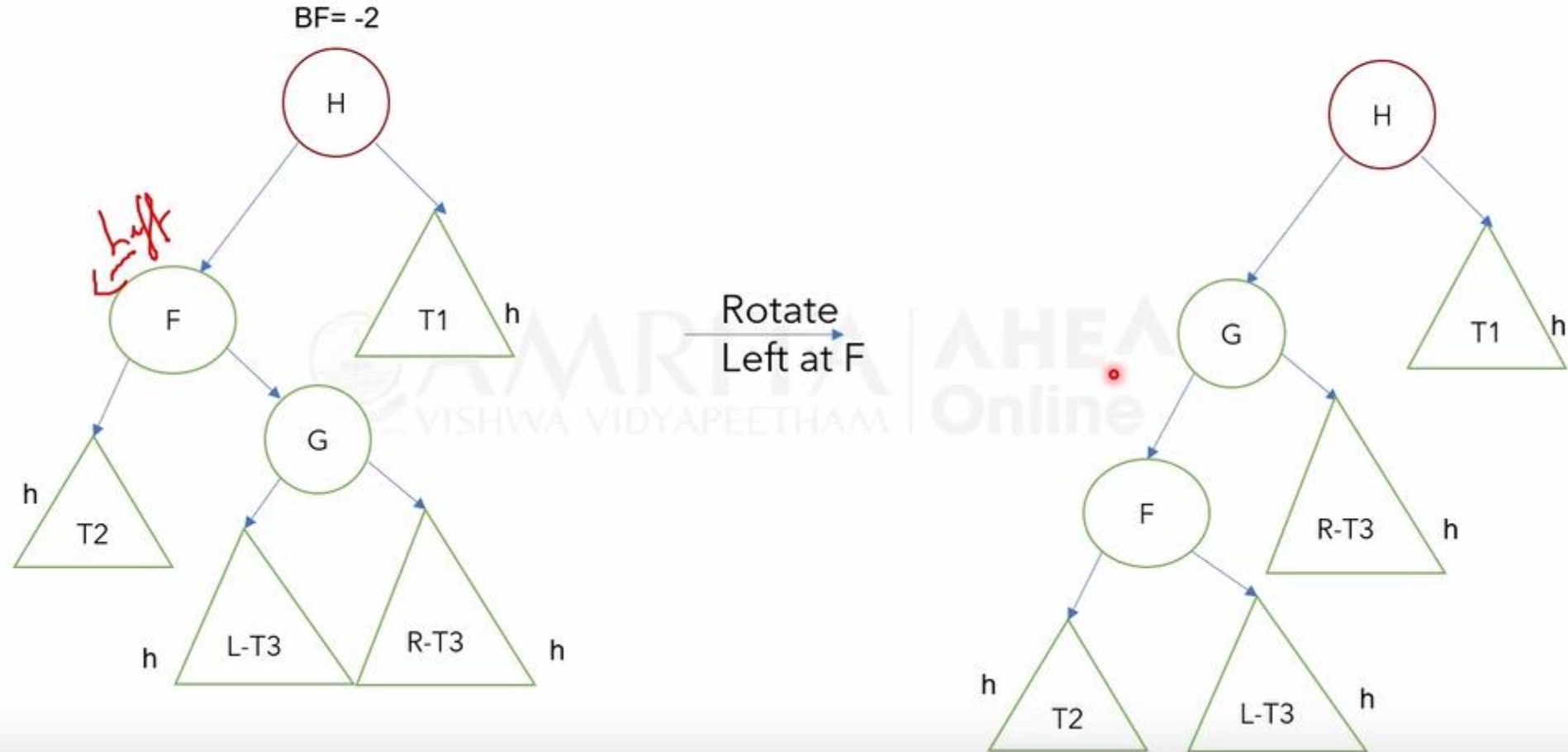
AVL Tree - Insertion

Case-4 Left-Right rotation



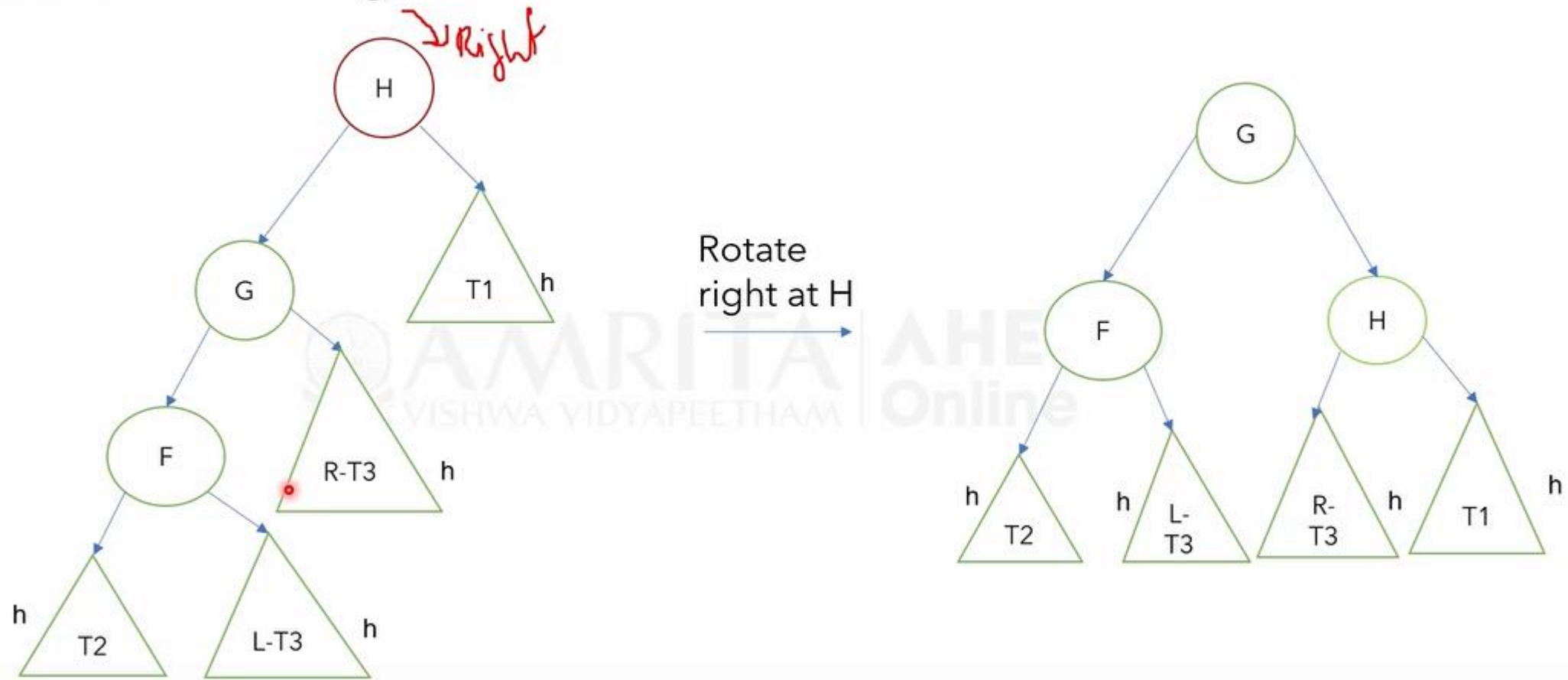
AVL Tree - Insertion

Case-4 Left-Right rotation



AVL Tree - Insertion

Case-4 Left -Right rotation



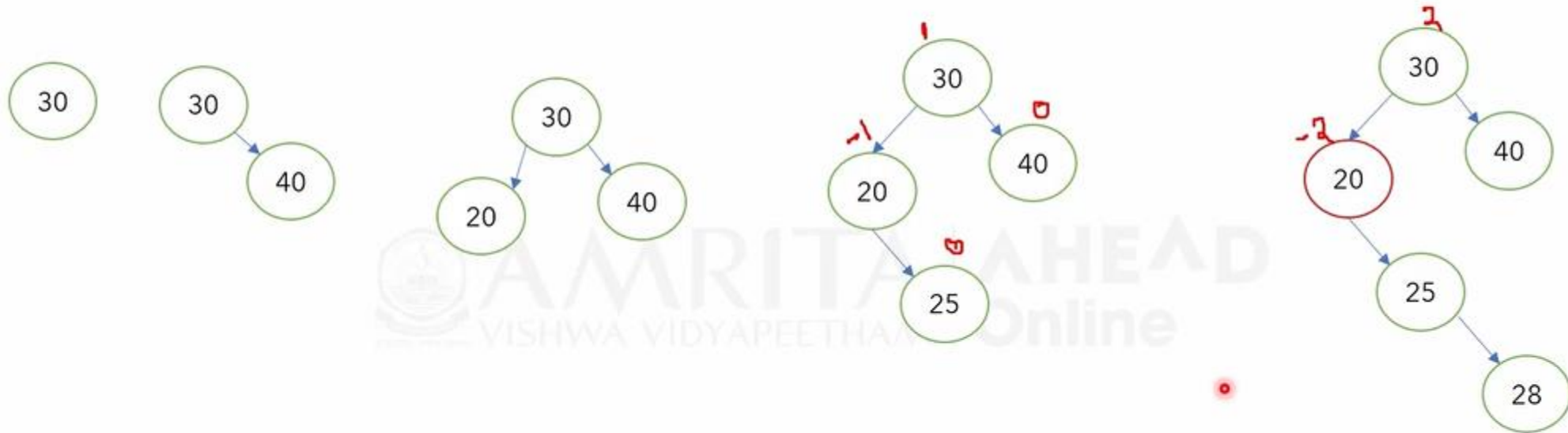
AVL Tree

Insert the following keys into an AVL tree 30, 40, 20, 25, 28, 21, 22



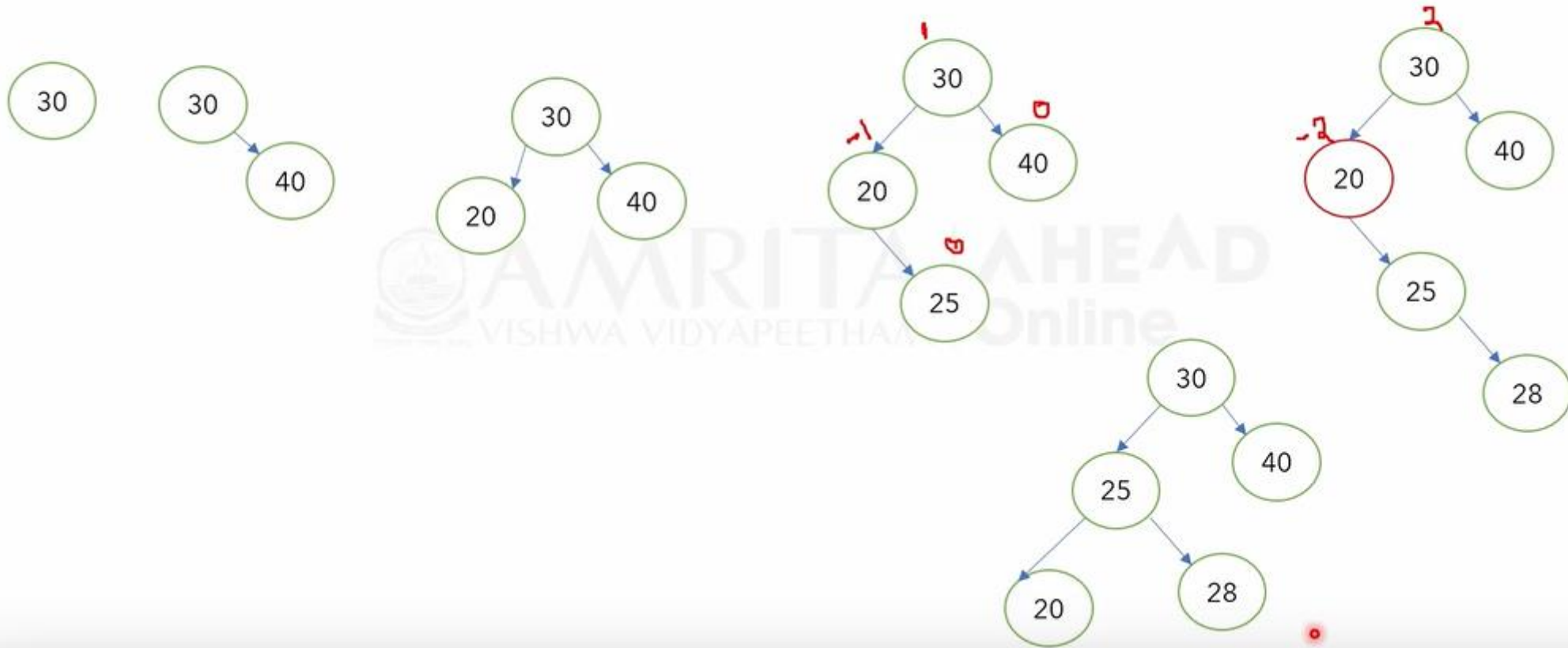
AVL Tree

Insert the following keys into an AVL tree 30, 40, 20, 25, 28, 21, 22



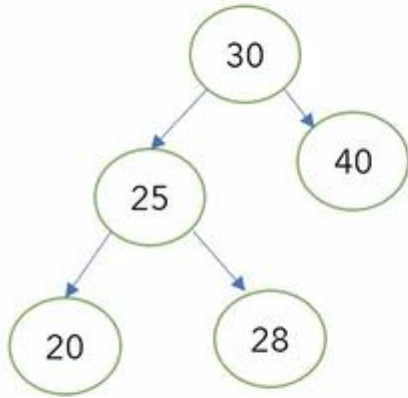
AVL Tree

Insert the following keys into an AVL tree 30, 40, 20, 25, 28, 21, 22



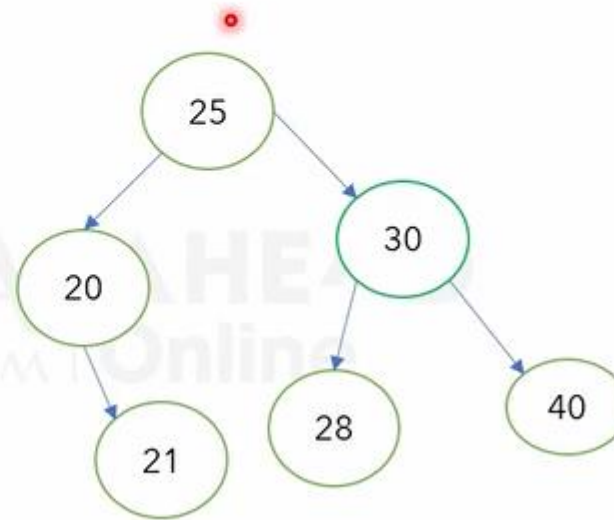
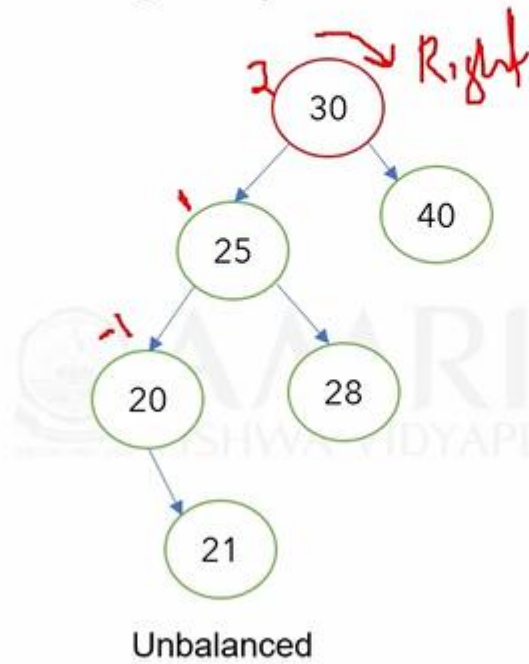
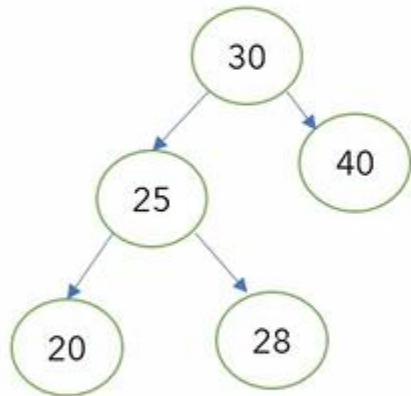
AVL Tree

Insert the following keys into an AVL tree 30, 40, 20, 25, 28, 21, 22



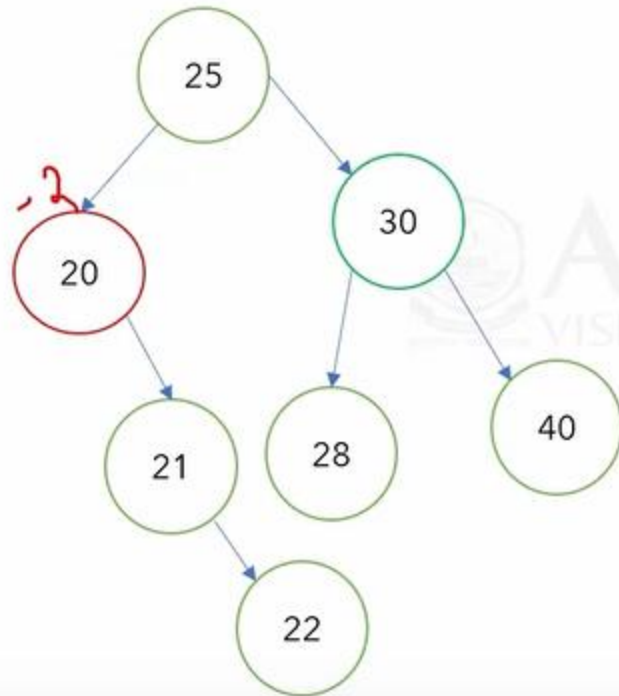
AVL Tree

Insert the following keys into an AVL tree 30, 40, 20, 25, 28, 21, 22



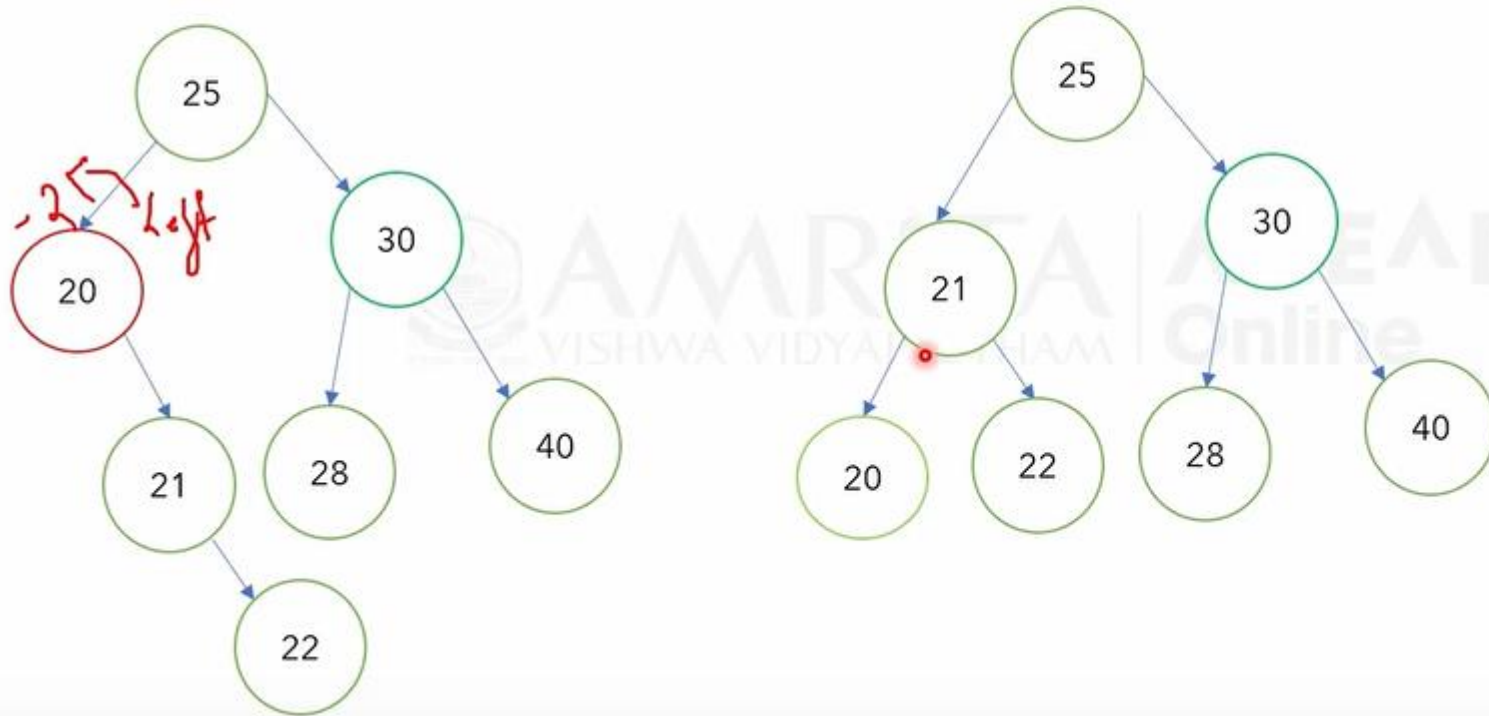
AVL Tree

Insert the following keys into an AVL tree 30, 40, 20, 25, 28, 21, 22



AVL Tree

Insert the following keys into an AVL tree 30, 40, 20, 25, 28, 21, 22



Summary

AVL Tree and its Insertion

Insertion - 4 cases

- Left child Left subtree - Single right rotation.
- Right child of Right subtree - Single left rotation.
- Left child of Right subtree - First Right and then Left Rotation.
- Right child of Left Subtree - First Left and then Right Rotation.